

Versatile Behavior Diffusion for Generalized Traffic Agent Simulation

Zhiyu Huang^{id}, *Member, IEEE*, Zixu Zhang, Ameya Vaidya, Yuxiao Chen^{id},
Jaime Fernández Fisac^{id}, *Member, IEEE*, and Chen Lv^{id}, *Senior Member, IEEE*

Abstract—Existing traffic simulation models often fall short in capturing the intricacies of real-world scenarios, particularly the interactive behaviors among multiple traffic participants, thereby limiting their utility in the evaluation and validation of autonomous driving systems. We introduce Versatile Behavior Diffusion (VBD), a novel traffic scenario generation framework based on diffusion generative models that synthesizes scene-consistent, realistic, and controllable multi-agent interactions. VBD achieves strong performance in closed-loop traffic simulation, generating scene-consistent agent behaviors that reflect complex agent interactions. A key capability of VBD is inference-time scenario editing through multi-step refinement, guided by behavior priors and model-based optimization objectives, enabling flexible and controllable behavior generation. Despite being trained on real-world traffic datasets with only normal conditions, we introduce conflict-prior and game-theoretic guidance approaches. These approaches enable the generation of interactive, customizable, or long-tail safety-critical scenarios, which are essential for comprehensive testing and validation of autonomous driving systems. Extensive experiments validate the effectiveness and versatility of VBD and highlight its promise as a foundational tool for advancing traffic simulation and autonomous vehicle development. Project website: <https://sites.google.com/view/versatile-behavior-diffusion>

Index Terms—Traffic simulation, multi-agent interaction, autonomous driving, scenario generation, diffusion model.

I. INTRODUCTION

SIMULATION testing is crucial for ensuring the safe deployment of autonomous driving systems. Traditionally, these assessments rely on replaying logged trajectories from driving datasets in simulation environments [1]. However, because other traffic participants in the replay do not

react to the autonomous vehicle’s actions, log-replay methods often fail to account for interactive traffic scenarios, such as pedestrians stepping into crosswalks or vehicles negotiating right-of-way at four-way stops. This limitation introduces a significant simulation-to-reality gap, where simulated test results may not translate to real-world performance, undermining the credibility of simulation-based testing.

To address this challenge, traffic simulators that incorporate reactive traffic behavior models have become increasingly popular. These models account for interactions between agents and their surroundings, enabling the synthesis of scenarios that better reflect the complex dynamics of real-world traffic. However, conventional heuristic-based traffic modeling approaches [2], [3] often fail to scale to the complexity of realistic traffic maneuvers. As a result, recent efforts [4], [5], [6], [7], [8] have shifted towards data-driven methods. These methods leverage large-scale driving datasets [9], [10], [11] and behavior cloning to model traffic behaviors. Despite these advancements, existing methods still focus on predicting marginal open-loop trajectories for individual agents, potentially resulting in a lack of scene consistency. This means that when multiple agents’ most likely trajectories are rolled out together, unnatural interactions and even collisions can occur [12], compromising the realism and reliability of simulated scenarios.

Furthermore, traffic simulators should offer versatility and enable controllable scenario generation to satisfy both explicit (e.g., goals or targets) and implicit (e.g., collision avoidance) requirements. Existing models are often specialized in generating nominal behaviors conditioned on high-level objectives, such as navigation goals [7]. However, scenario generation models should also address the long-tail challenge in most datasets, where rare and dangerous situations are underrepresented [13], potentially leading to inadequate testing in critical scenarios. Previous works in safety-critical traffic scenario generation [14], [15], [16], [17] have employed tailored training objectives to encourage adversarial behaviors (e.g., inducing collisions with the ego agent), which limits their ability to capture the nuances of real-world traffic interactions. For instance, a model trained to generate aggressive driving behaviors may struggle to produce scenarios where surrounding agents attempt to avoid such adversarial actions, highlighting the constraints imposed by its specialized learning objectives. Therefore, a more generalized traffic simulation framework is needed to produce a wide spectrum of scenarios accommodating various requirements at inference time without retraining. This flexibility is crucial in traffic simulation,

Received 3 August 2025; revised 14 November 2025; accepted 29 January 2026. This work was supported in part by the Nanyang Technological University (NTU)-DESAY SV Research Program under Grant 2018-0980; in part by the Agency for Science, Technology and Research (A*STAR), Singapore, through the MTC Individual Research Grant under Grant M22K2c0079; in part by the Ministry of Education (MOE), Singapore, through the Tier 2 Grant under Grant MOE-T2EP50222-0002; and in part by the Nanyang Technological University, under CoE Dean Interdisciplinary Grant. The Associate Editor for this article was X. Hu. (Zhiyu Huang and Zixu Zhang contributed equally to this work.) (Corresponding author: Chen Lv.)

Zhiyu Huang and Chen Lv are with the School of Mechanical and Aerospace Engineering, Nanyang Technological University, Singapore 639798 (e-mail: zhiyu001@e.ntu.edu.sg; lyuchen@ntu.edu.sg).

Zixu Zhang, Ameya Vaidya, and Jaime Fernandez Fisac are with the Department of Electrical and Computer Engineering, Princeton University, Princeton, NJ 08544 USA (e-mail: zixuz@princeton.edu; avaidya@princeton.edu; jfisac@princeton.edu).

Yuxiao Chen is with NVIDIA Research, Santa Clara, CA 95051 USA (e-mail: yuxiaoc@nvidia.com).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TITS.2026.3662886>, provided by the authors.

Digital Object Identifier 10.1109/TITS.2026.3662886

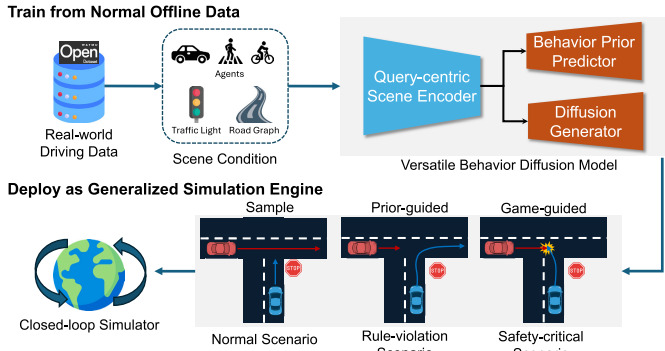


Fig. 1. Illustration of the VBD framework. The model consists of a query-centric transformer scene condition encoder, a marginal multi-modal behavior predictor for individual agents, and a diffusion generator for joint multi-agent behavior. In training, the model only utilizes commonly used traffic datasets. In testing, the model's versatility is showcased through various purposes enabled by simple sampling and different guidance structures (e.g., prior guidance for sophisticated target agents' behavior control and game-theoretic cost guidance for adversarial behaviors). VBD functions as a generalized simulation engine and operates effectively in a closed-loop traffic simulator.

as it allows users to assign different behavioral modes to agents, enables the evaluation of edge cases, and facilitates the exploration of long-tail situations, providing a more comprehensive and customizable simulation tool.

To overcome these limitations, we leverage diffusion models [18], [19], [20], [21], a class of generative modeling methods that gradually recover structured data from random noise. We develop **Versatile Behavior Diffusion (VBD)**, a flexible and scalable model for traffic behavior generation. VBD is capable of generating realistic, interactive, and controllable traffic scenarios for a large number of agents. As illustrated in Fig. 1, despite being trained only on commonly used public road traffic datasets, VBD can accommodate a variety of tasks, such as normal or adversarial behavior generation, counterfactual reasoning, and reactive simulation, through guided sampling combined with user-specified objectives. Our VBD model demonstrates strong performance on the Waymo Sim Agent benchmark and offers versatile generation capabilities. Our main contributions are summarized as follows:

- 1) We propose **Versatile Behavior Diffusion** for generalized traffic simulation, capable of generating realistic and controllable traffic agent behaviors with strong closed-loop simulation performance.
- 2) We demonstrate the versatility of VBD in simulations, showcasing its ability to generate diverse user-specified scenarios through flexible guidance schemes compatible with optimization objectives, behavior priors, and game-theoretic structures.
- 3) We conduct ablation studies to investigate the effects of various training and inference settings on multi-agent behavior generation.

II. RELATED WORK

A. Traffic Simulation

Traditional model-based or heuristic-based methods [2], [3] often fail to capture complex agent interactions in real-world scenarios, resulting in a significant gap between simulation and reality. Consequently, there has been a growing shift towards learning-based methods to enhance the realism and

interactivity of traffic simulations [22]. For example, BITS [23] employs imitation learning to simulate agent behaviors by inferring high-level intentions and replicating low-level driving actions. The socially-controllable behavior generation model proposed in [24] focuses on simulating social and interactive behaviors. Symphony [5] integrates learning-based policies with parallel beam search to further enhance realism. TrafficBots [7] introduces a shared policy conditioned on specific goals to generate configurable behaviors. Trajeglish [25] introduces a multi-agent sequence of motion tokens using a GPT-like encoder-decoder architecture. Another research direction focuses on generating safety-critical or adversarial scenarios to test the robustness of driving systems. STRIVE [16] generates challenging scenarios that can induce collisions with the ego planner through optimization in latent space. Similarly, AdvDO [14] and KING [15] utilize optimization-based methods to generate adversarial trajectories for robust planning. CAT [17] selects conflicting trajectories from agents' predicted behavior distributions to construct adversarial scenarios, supporting robust training of RL driving agents. However, most existing simulation models lack versatility, as they are specifically trained either for normal behaviors (maximum likelihood) or adversarial scenarios. We aim to bridge this gap by developing a generative framework that supports various tasks and requirements to enable generalizable, controllable, and realistic traffic simulation.

B. Behavior Prediction

Behavior or trajectory prediction models are widely utilized in both traffic simulations and driving policies [7], [8], [26]. Recent advances in learning-based behavior prediction models have significantly improved the accuracy of predicting single-agent motions [6], [27], [28] as well as joint multi-agent interactions at the scene level [29], [30], [31]. Leveraging large amounts of real-world data, these models are capable of generating multi-modal distributions of possible behaviors for multiple agents in a scene. However, these prediction models often face limitations in scalability and efficiency when applied to large-scale scenarios with numerous agents. Therefore, more powerful generative models, including diffusion models [32], [33] and autoregressive Transformer models [34], [35], have been increasingly applied in behavior prediction and generation tasks, demonstrating superior performance in large-scale predictions and simulations. Our proposed VBD model integrates multi-modal behavior prediction as high-level intention priors, enabling the estimation of realistic behavior distributions and sophisticated control over target agents' behaviors. This integration facilitates the diffusion model in generating specific scenarios based on the intended behaviors of particular agents.

C. Diffusion Models for Traffic Simulation

Diffusion models [19], [20], [21], [36] have gained widespread popularity in a variety of generative tasks, including image [37], audio [38], and video [39] generation. Recently, their application has been extended to traffic scenario generation. For instance, SceneDM [40] utilizes a diffusion model to generate consistent joint motions for all agents

in a scene. MotionDiffuser [32] employs a diffusion-based representation for joint multi-agent motion prediction and introduces a constrained sampling framework for controlled trajectory sampling. CTG [41] combines diffusion modeling with signal temporal logic (STL) rules to ensure compliance with traffic rules in generated trajectories. CTG++ [42] leverages Large Language Models (LLMs) to translate user queries into loss functions, guiding the diffusion model toward generating query-compliant scenarios. TRACE [43] proposes a guided diffusion model to generate future trajectories for pedestrians, employing analytical loss functions to impose trajectory constraints. Other models, such as DiffScene [44] and Safe-Sim [45], adopt guided diffusion techniques with adversarial optimization to generate safety-critical traffic scenarios. SceneDiffuser [46] proposed to use amortized diffusion to improve the quality and computational efficiency for closed-loop rollouts. While our work is closely related to these guided diffusion models, these approaches often remain limited to specific tasks and lack versatility across different simulation applications.

III. PRELIMINARIES

A. Problem Formulation

We formulate traffic scenario generation as a conditional trajectory optimization problem in a multi-agent setting. A scenario $\mathcal{S} = (\mathbf{x}, \mathbf{u}, \mathbf{c})$ consists of agent trajectories $\mathbf{x} = (\mathbf{x}^1, \dots, \mathbf{x}^A) \in \mathbb{R}^{A \times T \times D_x}$, corresponding control sequences $\mathbf{u} = (\mathbf{u}^1, \dots, \mathbf{u}^A) \in \mathbb{R}^{A \times T \times D_u}$, and scene context $\mathbf{c} \in \mathbb{R}^{D_c}$ that encodes maps, traffic signals, and initial agent states $\mathbf{x}_0$.

We can define the scenario generation task as a finite-horizon optimal control problem:

$$\begin{aligned} \min_{\mathbf{u}} \quad & \mathcal{J}_\theta(\mathbf{x}, \mathbf{u}; \mathbf{c}), \\ \text{s.t.} \quad & \mathbf{x}_0 = \mathbf{x}_0, \\ & \mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t), \quad \forall t \in 0, \dots, T-1 \end{aligned} \quad (1)$$

where f defines discrete-time joint dynamics, and $\mathcal{J}_\theta$ promotes realistic, feasible behaviors.

To model the distribution over control sequences $\mathbf{u}$ conditioned on context $\mathbf{c}$, we adopt the Maximum Entropy IRL [47] formulation, representing the target distribution as a Boltzmann distribution:

$$p(\mathbf{u}|\mathbf{c}) \approx p_\theta(\mathbf{u}|\mathbf{c}) := \frac{1}{Z_\theta} \exp(-\mathcal{J}_\theta(\mathbf{x}(\mathbf{u}), \mathbf{u}; \mathbf{c})), \quad (2)$$

which resembles an Energy-Based Model (EBM) [48], [49]. The learning objective is to maximize the conditional likelihood of $\mathbf{u}$ under the model p_θ :

$$\theta = \arg \max_{\theta} \mathbb{E}_{\mathcal{S} \sim \mathcal{D}} [\log p_\theta(\mathbf{u}|\mathbf{c})]. \quad (3)$$

Directly estimating $\nabla_{\mathbf{u}} \log p_\theta(\mathbf{u}|\mathbf{c})$ (the score function) is challenging due to limited coverage in low-likelihood regions. To address this, recent works have employed diffusion models [20], [21], [36], which gradually transform data samples into noise through a forward stochastic process and then learn to reverse the process. Training such a model can be interpreted as learning the gradient steps of trajectory optimization in the

score space, thus bridging generative modeling and control. As shown by [50], [51], and [52], reverse diffusion sampling in this context approximates stochastic gradient descent toward low-cost, high-likelihood trajectories. This insight allows diffusion models to serve as both generative models and implicit planners that solve the control problem defined in Eq. (1).

B. Denoising Diffusion Probabilistic Model (DDPM)

The discrete-time formulation of the forward diffusion process of DDPM [53] can be described as:

$$q(\tilde{\mathbf{u}}_1, \dots, \tilde{\mathbf{u}}_K | \mathbf{u}) := \prod_{k=1}^K q(\tilde{\mathbf{u}}_k | \tilde{\mathbf{u}}_{k-1}), \quad (4)$$

$$q(\tilde{\mathbf{u}}_k | \tilde{\mathbf{u}}_{k-1}) := \mathcal{N}(\tilde{\mathbf{u}}_k; \sqrt{1 - \beta_k} \tilde{\mathbf{u}}_{k-1}, \beta_k \mathbf{I}), \quad (5)$$

where $\beta_k \in (0, 1)$ is the k -th noise scale from a predefined noise scheduling, $\mathbf{u}$ is the clean control action sampled from data distribution, and $\tilde{\mathbf{u}}_k$ is the noisy action samples at diffusion step k . In the final step K , the data distribution approaches an isotropic Gaussian distribution $q(\tilde{\mathbf{u}}_K) \approx \mathcal{N}(\tilde{\mathbf{u}}_K; 0, \mathbf{I})$. Let $\alpha_k = 1 - \beta_k$ and $\bar{\alpha}_k = \prod_{i=0}^k \alpha_i$, we can directly sample $\tilde{\mathbf{u}}_k$ from data $\mathbf{u}$ without iterative diffusion via reparameterization trick:

$$\begin{aligned} \tilde{\mathbf{u}}_k &= \sqrt{\alpha_k} \tilde{\mathbf{u}}_{k-1} + \sqrt{1 - \alpha_k} \epsilon_{k-1} \\ &= \sqrt{\bar{\alpha}_k} \mathbf{u} + \sqrt{1 - \bar{\alpha}_k} \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}). \end{aligned} \quad (6)$$

The generation process is accomplished by learning to reverse the forward diffusion process based on context information $\mathbf{c}$. The reverse diffusion process starts with an isotropic Gaussian noise $q(\tilde{\mathbf{u}}_K)$ and can be expressed as:

$$p_\theta(\tilde{\mathbf{u}}_0, \dots, \tilde{\mathbf{u}}_K | \mathbf{c}) := q(\tilde{\mathbf{u}}_K) \prod_{i=1}^K p_\theta(\tilde{\mathbf{u}}_{k-1} | \tilde{\mathbf{u}}_k, \mathbf{c}), \quad (7)$$

$$p_\theta(\tilde{\mathbf{u}}_{k-1} | \tilde{\mathbf{u}}_k, \mathbf{c}) := \mathcal{N}(\tilde{\mathbf{u}}_{k-1}; \mu_\theta(\tilde{\mathbf{u}}_k, \mathcal{D}_\theta(\tilde{\mathbf{u}}_k, k, \mathbf{c})), \sigma_k^2 \mathbf{I}), \quad (8)$$

where μ_θ calculates the posterior mean of the noise at $k-1$ step from $\tilde{\mathbf{u}}_k$ and DDPM denoiser output $\mathcal{D}_\theta(\cdot)$, and σ_k is the standard deviation according to the fixed noise schedule.

Specifically, the denoiser $\mathcal{D}_\theta(\cdot)$ estimates the clean action trajectory sample $\hat{\mathbf{u}}_k$ from the current noisy sample $\tilde{\mathbf{u}}_k$, according to which the mean of the previous noisy sample $\tilde{\mathbf{u}}_{k-1}$ can be derived as follows:

$$\mu_k := \frac{\sqrt{\bar{\alpha}_{k-1}} \beta_k}{1 - \bar{\alpha}_k} \hat{\mathbf{u}}_k + \frac{\sqrt{\alpha_k} (1 - \bar{\alpha}_{k-1})}{1 - \bar{\alpha}_k} \tilde{\mathbf{u}}_k. \quad (9)$$

C. Controllable and Compositional Generation

We also want to generate scenarios that satisfy a specific user requirement y without retraining the model. For example, y can be defined as the goal or the reference path for individual agents, or it can describe a soft constraint, such as obeying the speed limit or avoiding collisions. We modify the optimization objective to: $\mathcal{J}_\theta(\mathbf{x}, \mathbf{u}; \mathbf{c}) + \mathcal{J}_y(\mathbf{x}, \mathbf{u}; \mathbf{c})$. Plugging into the EBM representation, we obtain a new conditional distribution: $p(\mathbf{u}|\mathbf{c}, y) \propto p(\mathbf{u}|\mathbf{c}) p(y|\mathbf{u}, \mathbf{c})$, where $p(\mathbf{u}|\mathbf{c})$ is the data distribution we approximated through generative modeling and $p(y|\mathbf{u}, \mathbf{c})$ is the likelihood of y . This immediately resembles the compositionality in EBM [50] and Classifier Guidance in

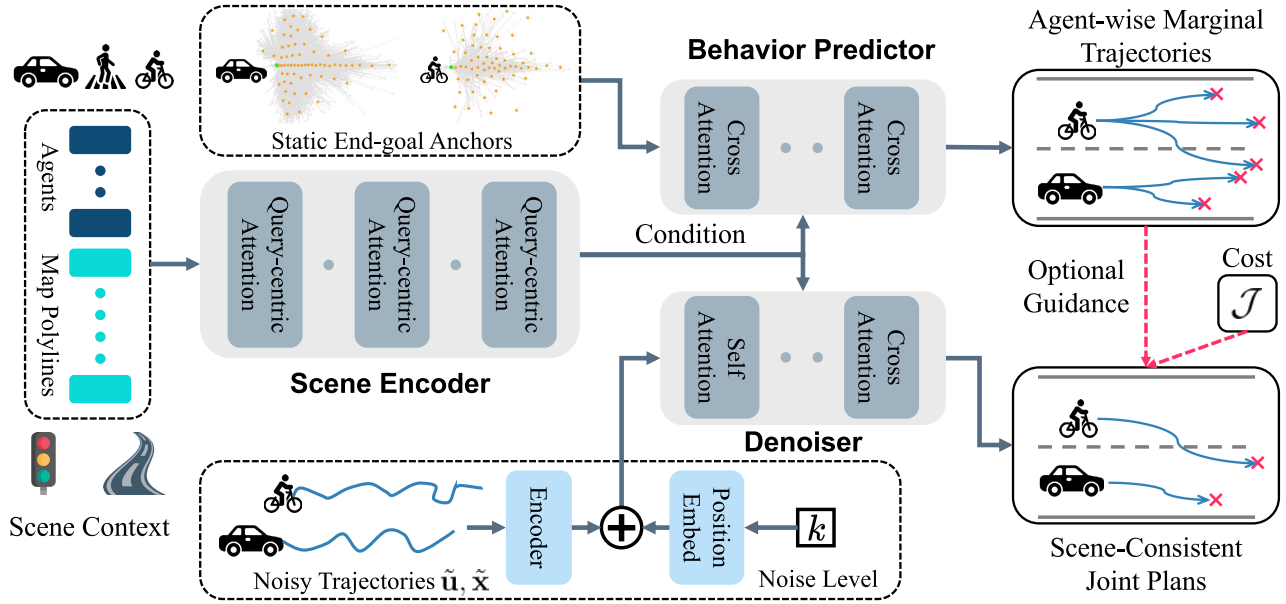


Fig. 2. Overview of the proposed VBD model structure. The input scenario context is encoded through a query-centric Transformer scene context encoder. The behavior predictor generates marginal multi-modal trajectories. The denoiser predicts the joint multi-agent future trajectories while attending to themselves and the condition tokens. During inference, the predicted behavior priors or user-defined model-based objectives $\mathcal{J}$ can be used to guide the denoising process to generate desired scenarios.

diffusion models [54]. Specifically, we can sample the reverse process with a conditional score function:

$$\begin{aligned} \nabla_{\tilde{\mathbf{u}}} \log p_k(\tilde{\mathbf{u}}|\mathbf{c}, y) &\approx \mathbf{s}_\theta(\tilde{\mathbf{u}}|\mathbf{c}, y, k) \\ &= \mathbf{s}_\theta(\tilde{\mathbf{u}}|\mathbf{c}, k) + \nabla_{\tilde{\mathbf{u}}} \log p_k(y|\tilde{\mathbf{u}}, \mathbf{c}), \end{aligned} \quad (10)$$

where $p_k(y|\tilde{\mathbf{u}}, \mathbf{c})$ is the likelihood of y given the noised action $\tilde{\mathbf{u}}$ at step k . It is important to note that $p_k(y|\tilde{\mathbf{u}}, \mathbf{c})$ is not equivalent to the likelihood of y in the data distribution $p(y|\mathbf{u}, \mathbf{c})$, therefore it is typically required to train a separate model [54], [55]. However, we can utilize a practical approximation to the gradient of the noised likelihood with the gradient of $\mathcal{J}_y$ for guidance, which enables flexible composition and controllability with additional objectives without training.

In the reverse diffusion process, the guidance is implemented by subtly altering the predicted mean of the model at each denoising step [41], [42]. Initially, we can directly approximate $\log p_k(y|\tilde{\mathbf{u}}, \mathbf{c}) \approx \log p(y|\tilde{\mathbf{u}}, \mathbf{c}) = \mathcal{J}_y(\mathbf{x}(\tilde{\mathbf{u}}), \tilde{\mathbf{u}}; \mathbf{c})$; therefore, the denoising mean is modified to impose guidance in the form of a score function $\mathcal{J}_y$ as:

$$\tilde{\mu}_k = \mu_k + \lambda \sigma_k \nabla_{\mu_k} \mathcal{J}_y(\mu_k), \quad (11)$$

where parameter λ controls the strength of the guidance.

However, calculating the gradient based on the mean of noisy actions is difficult and inaccurate, and thus manipulating the noise mean using the noisy gradient can result in errors and instability. To address this, we utilize an alternative approach [32] to approximate $\log p_k(y|\tilde{\mathbf{u}}, \mathbf{c}) \approx \mathcal{J}_y(\mathbf{x}(\mathcal{D}_\theta(\tilde{\mathbf{u}})), \mathcal{D}_\theta(\tilde{\mathbf{u}}); \mathbf{c})$. This means calculating the objective function using the one-step generation result from the denoiser rather than the noisy mean. The modification in the denoising step is expressed as:

$$\tilde{\mu}_k = \mu_k + \lambda \sigma_k \nabla_{\tilde{\mathbf{u}}_k} \mathcal{J}_y(\mathcal{D}_\theta(\tilde{\mathbf{u}}_k)), \quad (12)$$

where gradient $\nabla_{\tilde{\mathbf{u}}_k}$ is calculated with respect to the noisy actions $\tilde{\mathbf{u}}_k$ and necessitates differentiation through the denoiser.

IV. METHOD

A. Model Structure

The **Versatile Behavior Diffusion (VBD)** model consists of three main components, as illustrated in Fig. 2. The scene encoder $\mathcal{E}_\phi : \mathbf{c} \mapsto \hat{\mathbf{c}}$ encodes the scene context $\mathbf{c}$ into its latent representation $\hat{\mathbf{c}}$ using query-centric attention Transformers [30]. Leveraging rich scene context information from the encoder, the denoiser $\mathcal{D}_\theta : (\hat{\mathbf{c}}, \tilde{\mathbf{u}}, k) \mapsto \hat{\mathbf{u}}$ directly predicts a joint clean control sequence $\hat{\mathbf{u}}$ from $\hat{\mathbf{c}}$ and noised control $\tilde{\mathbf{u}}$ at step k . The behavior predictor $\mathcal{P}_\psi : (\hat{\mathbf{c}}, \{\zeta^i\}_{i=1}^M) \mapsto \{\text{Cat}_M(\hat{\mathbf{u}}^a, \hat{\omega}^a)\}_{a=1}^A$ predicts an M -mode marginal categorical trajectory distribution of *each agent* from $\hat{\mathbf{c}}$ with the help of a set of representative static end-point anchors $\{\zeta^i\}_{i=1}^M$ extracted from data [30]. All three modules utilize a stack of query-centric self-attention and cross-attention blocks.

Scene Context. The scene conditions are divided into three categories: agents $\mathbf{c}_a \in \mathbb{R}^{A \times T_h \times D_a}$, map polylines $\mathbf{c}_{m,pl} \in \mathbb{R}^{M_l \times M_p \times D_p}$, and traffic lights $\mathbf{c}_{m,tl} \in \mathbb{R}^{M_l \times D_t}$. Here, T_h denotes the number of historical steps, M_l the number of polylines, M_p the number of waypoints per polyline, M_t the number of traffic lights, and $M_l + M_t = M$ represents the combined count of map elements. The feature sizes for agents, polylines, and traffic lights are represented by D_a , D_p , and D_t , respectively. The agent history tensor records each agent's historical state, including x, y coordinates, heading angle (ψ), velocities (v_x, v_y), and bounding box dimensions (l, w, h), along with the agent type. Each map polyline, comprising M_p waypoints, includes attributes such as x, y coordinates, direction angle, the traffic light state controlling the lane, and lane type.

The traffic lights tensor encompasses x, y coordinates of stop points and the state of each traffic light. Before encoding, positional attributes of all elements are converted into their local coordinate systems; for agents, the reference point is their last recorded state, and for map polylines, it is the location of the first waypoint.

Scene Encoder. We encode the agent history tensor utilizing a shared GRU network, which is then combined with the agent type embedding. For map polylines, an MLP is employed for encoding, followed by max-pooling along the waypoint axis; for traffic lights, we encode only their light status using an MLP. These tensors are then concatenated to form the initial scene encoding. The initial scene encoding is further processed using $L_{\mathcal{E}}$ query-centric Transformer layers to symmetrically encode the interrelationships among scene components, resulting in a uniform dimension for scene features $\hat{\mathbf{c}} \in \mathbb{R}^{(A+M) \times D}$. In this approach, each scene element is translated into its local coordinate system and encoded with query-centric features, and the relative position of each pair of scene elements is calculated and encoded as edge attributes.

Denoiser. The denoiser $\mathcal{D}_{\theta}(\tilde{\mathbf{u}}(k), k, \hat{\mathbf{c}})$ receives as input the noised agent actions $\tilde{\mathbf{u}}(k) \in \mathbb{R}^{A \times T \times 2}$ from the clean actions $\mathbf{u}(0) = f^{-1}(\mathbf{x}(0))$, which are computed using an inverse dynamics model f^{-1} from the ground truth states $\mathbf{x}(0) \in \mathbb{R}^{A \times T \times 4}$. Each agent's action at every timestep $u = [\dot{v}, \dot{\psi}]^T$ consists of acceleration and yaw rate, while the state comprises coordinates, heading, and velocity (x, y, ψ, v) . The denoiser also takes the noise level k and the encoded scene conditions $\hat{\mathbf{c}}$ as inputs. It encompasses several Transformer layers with both self-attention and cross-attention mechanisms.

Initially, the noised actions $\tilde{\mathbf{u}}_k$ are converted to noised states $\tilde{\mathbf{x}}(k) = f(\tilde{\mathbf{u}}(k))$, encoded via an MLP, and concatenated with the noise level embedding. Then, a self-attention Transformer module is employed to model the joint distribution of future plans across agents. To maintain closed-loop rollout causality, a causal relationship mask [56] is used in the self-attention module, ensuring that information from future timesteps cannot be utilized at the current timestep. Furthermore, a cross-attention Transformer module is used to model the scene-conditional distribution by relating the noisy trajectories to the encoded scene conditions. This decoding block can be repeated several times, and the final embedding is fed into an MLP decoder to predict the denoised actions $\hat{\mathbf{u}}(0)$ and clean states $\hat{\mathbf{x}}(0)$ through a dynamics function f .

Behavior Predictor. The behavior predictor $\mathcal{P}_{\psi}$ generates the marginal distributions of possible behaviors for individual agents by directly decoding from the encoded scene conditions $\hat{\mathbf{c}}$. The predictor is composed of $L_{\mathcal{P}}$ cross-attention Transformer decoder layers. To accurately predict the probabilities of possible goals, we utilize type-wise static anchors (end goals) $\{\zeta^i\}_{i=1}^M$ extracted from data [30] as the modality query inputs to the initial Transformer decoder layer. The predictor iteratively refines its predictions through several decoding layers and finally generates M trajectories over the future time horizon for each agent along with their associated scores, represented as $\{\text{Cat}_M(\hat{\mathbf{u}}^a, \hat{\omega}^a)\}_{a=1}^A \in \mathbb{R}^{A \times M \times T \times 5}$. To ensure the kinematic feasibility of these goals, the model predicts action trajectories, which are converted into state trajectories using

the same dynamics function f . Each waypoint in the state trajectory contains the state (x, y, ψ, v) and the probability of the trajectory $\hat{\omega}$.

B. Model Training

We implement a multi-task learning framework that concurrently trains the encoder, denoiser, and predictor components of our model. To train the denoiser, we aim to minimize the denoising loss:

$$\mathcal{L}_{\mathcal{D}_{\theta}} = \mathbb{E}_{S \sim p, k \sim \mathcal{U}(0, K)} \mathbb{E}_{\tilde{\mathbf{u}} \sim p_k(\cdot | \mathbf{u})} [\lambda(k) \mathcal{SL}_1(\hat{\mathbf{x}}(\mathcal{D}_{\theta}(\hat{\mathbf{c}}, \tilde{\mathbf{u}}, k)) - \mathbf{x})], \quad (13)$$

which is defined as the Smooth L1 loss between ground-truth trajectories $\mathbf{x}$ and the trajectories $\hat{\mathbf{x}}$ rolled out from $\hat{\mathbf{u}}$. Essentially, the denoiser is trained to recover the clean trajectories under various noise levels. At each training step, noise level k is sampled and applied to corrupt the ground-truth trajectories, and the denoiser is optimized to predict the denoised trajectories from the corrupted trajectories. Since the model predicts scene-level joint trajectories, all agent trajectories are affected by the same noise level. The design of the cost function plays a crucial role in generation performance.

In addition, we utilize a log noise schedule defined as:

$$\bar{\alpha}_k = \frac{f(k)}{f(0)}, f(k) = \log \left(\frac{K + K\delta}{k + K\delta} \right), k = [0, 1, \dots, K], \quad (14)$$

where K is the maximum diffusion step and δ is a scale factor that controls the variance changing rate of the log noise schedule. This schedule ensures that the signal-to-noise ratio (SNR) remains sufficiently low, preventing degradation in inference performance due to shortcut learning. Empirically, we found the performance of this noise schedule is significantly better than the Cosine noise scheduler [53].

Training a denoiser with the scene encoder directly can be unstable, partially because the denoiser focuses on structured data from the noisy trajectories rather than the information from context encoding. To address this issue, we suggest incorporating an additional task in the model to predict marginal multi-modal trajectories, which can more effectively attend to the context encoding. This setting not only stabilizes training and enhances the overall learning performance but also provides behavior priors for individual agents. To train the behavior predictor $\mathcal{P}_{\psi}$, we follow the multi-trajectory-prediction (MTP) loss setting. This involves selecting the best-predicted mode m^* that most closely matches the ground truth trajectory of each agent. To determine the best-predicted mode for an agent, the following criterion is applied:

$$m^* = \begin{cases} \arg \min_i \|\mathbf{ac}^i - \mathbf{x}_T\|, & \text{if } \mathbf{x}_T \text{ is valid,} \\ \arg \min_i \left\| \sum_t (\hat{\mathbf{x}}_t^i - \mathbf{x}_t) \right\|, & \text{otherwise,} \end{cases} \quad (15)$$

where $\mathbf{ac}^i$ is the static anchor point, $\mathbf{x}_t$ is the ground-truth point of the trajectory, and $\hat{\mathbf{x}}_t^i$ is the predicted trajectory point. This means that if the ground-truth trajectory endpoint is invalid, the predicted trajectory with the smallest average displacement error is selected; otherwise, the trajectory corresponding to the closest anchor point is selected.

Subsequently, trajectories are chosen from the multi-modal predictions based on the indices m^* , and the Smooth $L1$ loss is computed between these selected trajectories and the ground-truth trajectories. For the training of the scoring function, cross-entropy loss is utilized, comparing the predicted logits with the given modes. The prediction loss is formulated as:

$$\mathcal{L}_{\mathcal{P}_\psi} = \mathbb{E}_{\mathcal{S} \sim p} \left[\sum_{a=1}^A \mathcal{SL}_1(\hat{\mathbf{x}}(\hat{\mathbf{u}}^{a,m^*}) - \mathbf{x}^a) + \beta \text{CE}(m^*, \hat{\omega}^a) \right], \quad (16)$$

where β is a hyperparameter. Note that this loss function is computed marginally, and time steps lacking ground-truth data or invalid agents are excluded from the loss calculation.

The total loss function for the multi-task learning model is formulated as:

$$\mathcal{L} = \mathcal{L}_{\mathcal{D}_\theta} + \gamma \mathcal{L}_{\mathcal{P}_\psi}, \quad (17)$$

where γ is a hyperparameter to balance different tasks.

V. EXPERIMENTAL SETUP

A. System Dynamics

The diffusion model operates in the action space, and we assume that there is a dynamic function that can translate actions to physical states $\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t)$. We utilize unicycle dynamics as the system dynamics $\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t)$ to roll out actions into states. This model is shared across all types of agents, including vehicles, pedestrians, and cyclists. The current state of an agent is defined by its global coordinates (x, y) , yaw angle ψ , and velocities v_x, v_y . Given the action of an agent, including acceleration $\dot{v}$ and yaw rate $\dot{\psi}$, and the time length for one step Δt , the next-step state of the agent is calculated using the following forward dynamics f :

$$\begin{aligned} x(t+1) &= x_t + v_x(t)\Delta t, \\ y(t+1) &= y_t + v_y(t)\Delta t, \\ \psi(t+1) &= \psi(t) + \dot{\psi}\Delta t, \\ v(t+1) &= \sqrt{v_x(t)^2 + v_y(t)^2} + \dot{v}\Delta t, \\ v_x(t+1) &= v(t+1) \cos \psi(t+1), \\ v_y(t+1) &= v(t+1) \sin \psi(t+1). \end{aligned} \quad (18)$$

Since each operation in the dynamics function is differentiable, it can be integrated as a layer in the network to convert predicted actions into states. Furthermore, we employ the inverse dynamics function f^{-1} to calculate actions from ground-truth states, which is formulated as:

$$\begin{aligned} \dot{v}(t) &= \frac{v(t+1) - v(t)}{\Delta t}, \\ v(t) &= \sqrt{v_x(t)^2 + v_y(t)^2}, \\ \dot{\psi}(t) &= \frac{\psi(t+1) - \psi(t)}{\Delta t}. \end{aligned} \quad (19)$$

Throughout our experiment, we normalize the acceleration by 1 m/s^2 and the yaw rate by 0.15 rad/s .

The unicycle dynamics used for mapping between actions and states may introduce small discrepancies between reconstructed and ground-truth trajectories. However, our empirical

analysis shows that this effect is minor in practice. Across 40K agent trajectories from the Waymo Motion dataset, the average displacement error between 8-second reconstructed and ground-truth trajectories is 0.221 m, with a final displacement error of 0.511 m. These results show that the inverse dynamics model yields sufficiently accurate state-action reconstructions. In our framework, inverse dynamics is used only to obtain an approximate control-space representation, while supervision is applied on state trajectories after forward rollout through the differentiable forward dynamics function. This rollout-based supervision mitigates potential mismatches, ensuring consistent trajectory generation.

B. Dataset

We conduct the experiments on the Waymo Open Motion Dataset (WOMD) [57], which comprises 486,995 nine-second logged real-world traffic scenarios for training and 44,097 scenarios for validation. The dataset provides trajectories for all agents and corresponding vectorized maps for each scenario. In the Waymo Sim Agents task [11], we evaluate our model's closed-loop simulation performance across 44,920 testing scenarios. In addition, we select 500 scenarios from the WOMD Validation Interactive split for additional experiments and ablation studies.

We choose only the WOMD instead of nuScenes [9] and Argoverse [58] for the following reasons. First, nuScenes and Argoverse are primarily designed for perception and short-term prediction benchmarks, which are insufficient for extended, interactive multi-agent simulation. Second, neither provides a standardized closed-loop evaluation framework that can simulate and replay the behavior of all traffic participants over the full duration of each scenario. In contrast, we use the Waymax environment [59] built on WOMD, which supports detailed, longer-horizon (8 seconds) closed-loop simulation of every agent in the scene, allowing us to rigorously assess multi-agent interaction dynamics.

C. Implementation Details

Model Inputs. For the scene conditions, we consider $A = 64$ agents, $M_l = 256$ map polylines (each containing $M_p = 30$ waypoints), and $M_t = 16$ traffic lights in the scene. The flexibility of Transformer blocks allows VBD to be adapted to any number of agents during inference. VBD generates $T = 80$ steps of future control sequences with step size 0.1 s based on only the current state and up to 1 s of past trajectories ($T_h = 11$). Empirically, we observe that discarding entire past trajectories and only keeping the current state during inference leads to better closed-loop performance by addressing the causal confusion issue in closed-loop testing [60].

Model Details. The scene encoder contains $L_{\mathcal{E}} = 6$ query-centric-attention Transformer layers, and the embedding dimension is $D = 256$. The behavior predictor comprises $L_{\mathcal{P}} = 4$ cross-attention Transformer layers and generates 64 possible trajectories for each agent along with respective probability estimates. The denoiser includes two decoding blocks with four Transformer layers in total. Action sequences are effectively shortened to $T_f = 40$ from T , by repeating

TABLE I
TESTING RESULTS ON THE 2024 WAYMO SIM AGENTS BENCHMARK

Agent Policy	Realism Meta Metric ($\uparrow$)	Kinematic Metric ($\uparrow$)	Interactive Metric ($\uparrow$)	Map-based Metric ($\uparrow$)
SMART	0.759	0.476	0.804	0.767
BehaviorGPT	0.747	0.433	0.800	0.764
GUMP	0.743	0.478	0.789	0.740
MVTE	0.730	0.450	0.771	0.743
SceneDiffuser	0.703	0.429	0.749	0.767
TrafficBotsV1.5	0.699	0.430	0.711	0.744
VBD (Ours)	0.720	0.417	0.782	0.721

actions over two time steps, which considerably reduces computational demands while ensuring high accuracy.

Training Details. The proposed log variance schedule is adopted in the diffusion process, employing $K = 50$ diffusion steps, $\bar{\alpha}_{min} = 1e - 9$, and $\delta = 0.0031$. For model training, the hyperparameters include a total loss function coefficient $\gamma = 0.5$ and a predictor loss coefficient $\beta = 0.05$. The model is trained using an AdamW optimizer with a weight decay of 0.01. The initial learning rate is set at 0.0002 and decays by 0.02 every 1,000 training steps, and a linear warm-up is employed for the first 1,000 steps. The total number of epochs for training is 16. Gradient clipping is implemented with a norm limit set to 1.0. The training of the model utilizes BFloat16 Mixed precision on 8 NVIDIA L40 GPUs, with an effective batch size of 96 across all GPUs.

VI. CLOSED-LOOP TRAFFIC SIMULATION

We demonstrate the VBD’s capability and generality in generating interactive and realistic traffic scenarios through a closed-loop evaluation. For each scene, we initialize agent states from WOMD and leverage the Waymax simulator [59] for scenario roll-out. At each time of inference, VBD generates 8-second trajectories for all agents in the scene via the standard DDPM denoising scheme. All agents replan at 1 Hz using a receding horizon approach.

We evaluate the scalability of VBD’s generation quality and interaction modeling capabilities using the Waymo Sim Agents Benchmark. In each scene, we are required to simulate 32 independent closed-loop rollouts of future scenarios for up to 128 agents over an 8-second horizon, conditioning on their trajectories from the preceding 1-second and the corresponding map contexts. Throughout the evaluation, VBD controls up to 64 agents surrounding the labeled self-driving vehicle in each scene, while a constant velocity policy controls other agents. By adopting this approach, we significantly reduce computational requirements when sampling millions of scenarios, leveraging the fact that most WOMD scenes contain fewer than 64 dynamic agents, with the remainder being stationary. We only utilize the current state of all agents by dropping out their past trajectories to overcome the causal confusion. We follow the official evaluation metrics of the Waymo Sim Agents benchmark [11], encompassing kinematic, interactive, and map-based metrics, and a meta realism metric is calculated as a weighted sum of these features.

In Table I, we compare the performance of VBD with state-of-the-art generative models, including BehaviorGPT [35],

SMART [61], and GUMP [62], behavior cloning models, such as MVTE [63] and TrafficBotsV1.5 [64], and concurrent diffusion-based model SceneDiffuser [46]. We demonstrate that the simulation performance of the VBD model is comparable to state-of-the-art generative models and outperforms other motion prediction models. In addition, we observe better simulation realism metrics compared to the SceneDiffuser [46] baseline. Notably, our VBD model achieves this with fewer parameters than autoregressive generation models, achieving a balance between performance and computational efficiency. We present a selection of qualitative simulation results in Fig. 3, showcasing the model’s ability to generate interactive and realistic traffic scenarios. The results in the Waymo Sim Agents benchmark suggest that our VBD model is reliable at generating realistic agent interactions through its joint multi-agent diffusion policy. Closed-loop simulation videos can be found on the Project Website.

While the proposed VBD model achieves strong results on the Sim Agent benchmark, a performance gap remains compared to autoregressive models such as SMART and BehaviorGPT. We believe this discrepancy largely comes from architectural and training factors. In particular, the map representation and encoder design in VBD can be further improved to better capture spatial dependencies between the road map and agents’ future trajectories, and increasing the number of denoising steps yields limited benefit under the realism metric. Despite these factors, the VBD model offers distinct advantages, including joint multi-agent consistency and flexible inference-time controllability, enabling user-guided behavior generation such as adversarial maneuvers.

VII. CONTROLLABLE SCENARIO GENERATION

A key strength of the VBD model lies in its capacity to generate customizable traffic scenarios without requiring retraining. This flexibility stems from the guided generation, allowing the model to adapt behaviors based on user-specified objectives, such as avoiding collisions or creating adversarial scenarios. By integrating these objectives into the denoising process, the VBD model provides a versatile and potent tool for simulating diverse traffic scenarios on demand. As outlined in Algorithm 1, users can define cost objectives $\mathcal{J}_y$, enabling compositional optimization via the classifier guidance method [54]. This involves iteratively refining the noisy $\tilde{\mathbf{u}}$ using both denoiser outputs and gradients of the form $\nabla_{\tilde{\mu}_k} \mathcal{J}_y(\mathcal{D}\theta(\tilde{\mu}_k))$. Throughout this process, the denoiser ensures scenario realism, while the user’s objective encourages the model to explore desired modes. Subsequent sections will demonstrate the application of VBD’s capabilities across various simulation tasks. Specifically, our guided generation tasks utilize $N_g = 5$ gradient steps and a fixed scaling parameter $\lambda = 0.1$ during all denoising operations.

We showcase the controllable scenario generation capabilities of our model in fulfilling various simulation tasks. To test this, we leverage the same subset of 500 WOMD interactive validation scenarios. Within each scene, we select the 32 agents closest to the labeled ego agent for evaluation, excluding all others. Our simulations span an 8-second horizon with a replanning frequency of 1 Hz, utilizing metrics

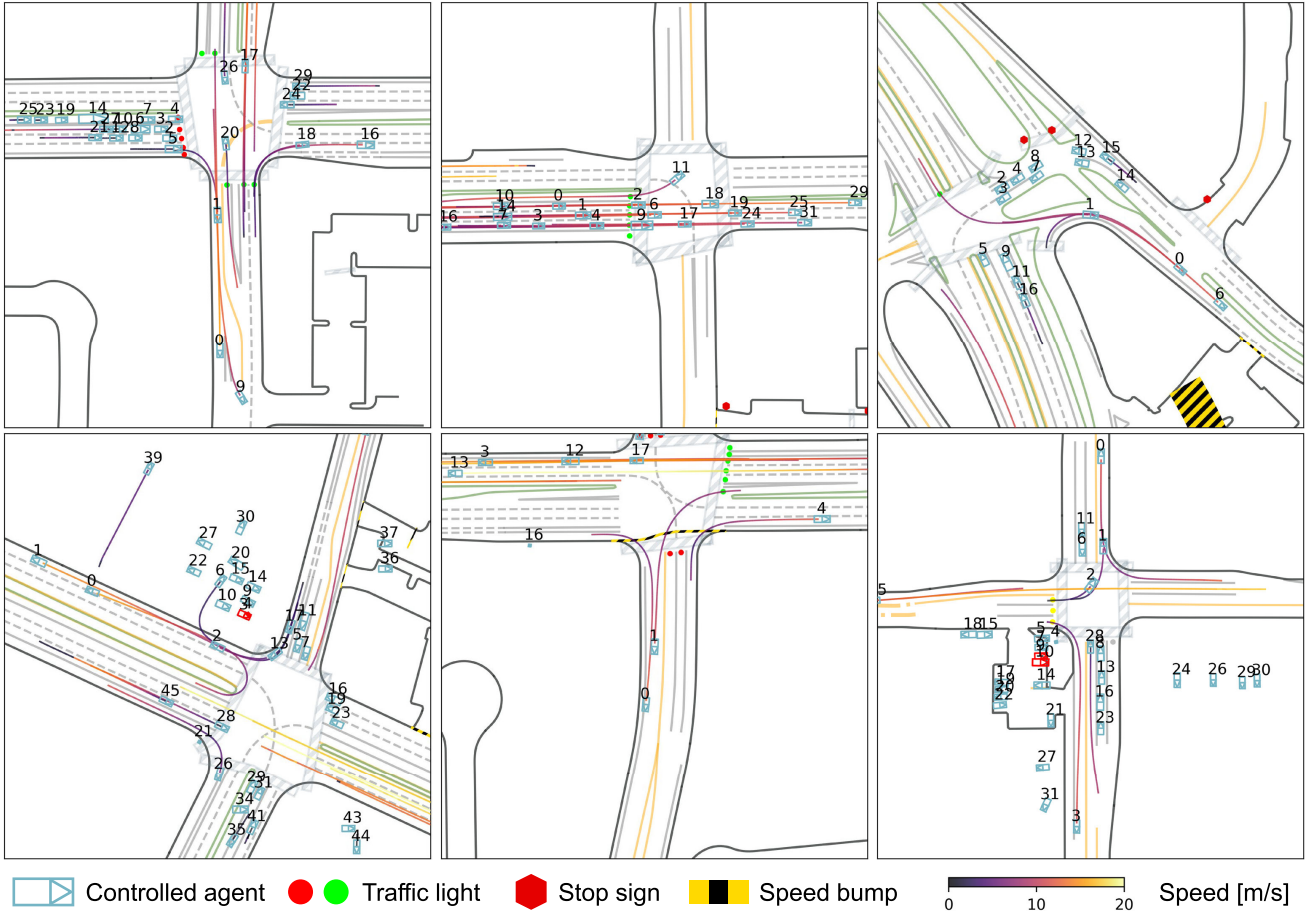


Fig. 3. Performance of our VBD model on the Waymo Sim Agents task. The multi-agent diffusion policy is capable of controlling a large number of agents in an interactive and map-adherent manner for traffic simulation.

Algorithm 1 Guided Sampling With Objective Function

Require: Denoiser $\mathcal{D}_\theta$, objective function $\mathcal{J}_y$, diffusion steps K , gradient steps N_g , scaling parameter λ , standard deviation σ_k

- 1: $\tilde{\mathbf{u}}_K \sim \mathcal{N}(0, \mathbf{I})$ ▷Sample initial trajectory
- 2: **for** $k \leftarrow K$ to 1 **do**
- 3: $\hat{\mathbf{u}} \leftarrow \mathcal{D}_\theta(\tilde{\mathbf{u}}_k, k, \mathbf{c})$ ▷Predict denoised control sequence
- 4: $\tilde{\mu}_k \leftarrow \frac{\sqrt{\alpha_k}(1-\bar{\alpha}_{k-1})}{1-\bar{\alpha}_k} \tilde{\mathbf{u}}_k + \frac{\sqrt{\bar{\alpha}_{k-1}}\beta_k}{1-\bar{\alpha}_k} \hat{\mathbf{u}}$ ▷Calculate unguided posterior $\tilde{\mu}_k$
- 5: **for** $i \leftarrow 1$ to N_g **do**
- 6: $\tilde{\mu}_k \leftarrow \tilde{\mu}_k + \lambda \sigma_k \nabla_{\tilde{\mu}_k} \mathcal{J}_y(\mathcal{D}_\theta(\tilde{\mu}_k))$ ▷Guidance gradient
- 7: **end for**
- 8: $\tilde{\mathbf{u}}_{k-1} \sim \mathcal{N}(\tilde{\mu}_k, \sigma_k^2 \mathbf{I})$ ▷Sample previous-step noised control sequence
- 9: **end for**
- 10: **Return:** Final control sequence $\mathbf{u} \leftarrow \tilde{\mathbf{u}}_0$

from the Waymax simulator, including off-road incidents, collisions, wrong-way occurrences, kinematic infeasibilities, and log divergences. Furthermore, we analyze all quantitative results across three distinct random seeds, averaging outcomes to assess the generality and consistency.

We employ the following metrics to thoroughly test our proposed method quantitatively:

- *Off-road*: A binary metric indicating whether a vehicle drives off the road, determined by its position relative to oriented road graph points.
- *Collision*: A binary metric identifying if collisions happen between agents.
- *Wrong-way*: A binary metric measuring whether a vehicle deviates from its intended driving direction.
- *Kinematic infeasibility (Kin.)*: A binary metric assessing whether a vehicle's movement is kinematically plausible.
- *Log divergence (ADE and FDE)*: These metrics quantify the deviation from ground truth behavior using displacement error. Average Displacement Error (ADE) is the L2 distance between an agent's simulated and ground-truth positions at each time step. Final Displacement Error (FDE) is the L2 distance error at $t = 8$ seconds.
- *Minimum Log divergence (minADE and minFDE)*: These metrics quantify the minimum deviation from ground truth behavior using displacement error among all generated scenarios for each scene.

A. Composition With Behavior Priors

Sampling diverse outputs from a conditional diffusion model is challenging, especially when denoising strongly relies on the context information [65]. On the other hand, the behavior predictor in VBD captures the multi-modal trajectories of indi-

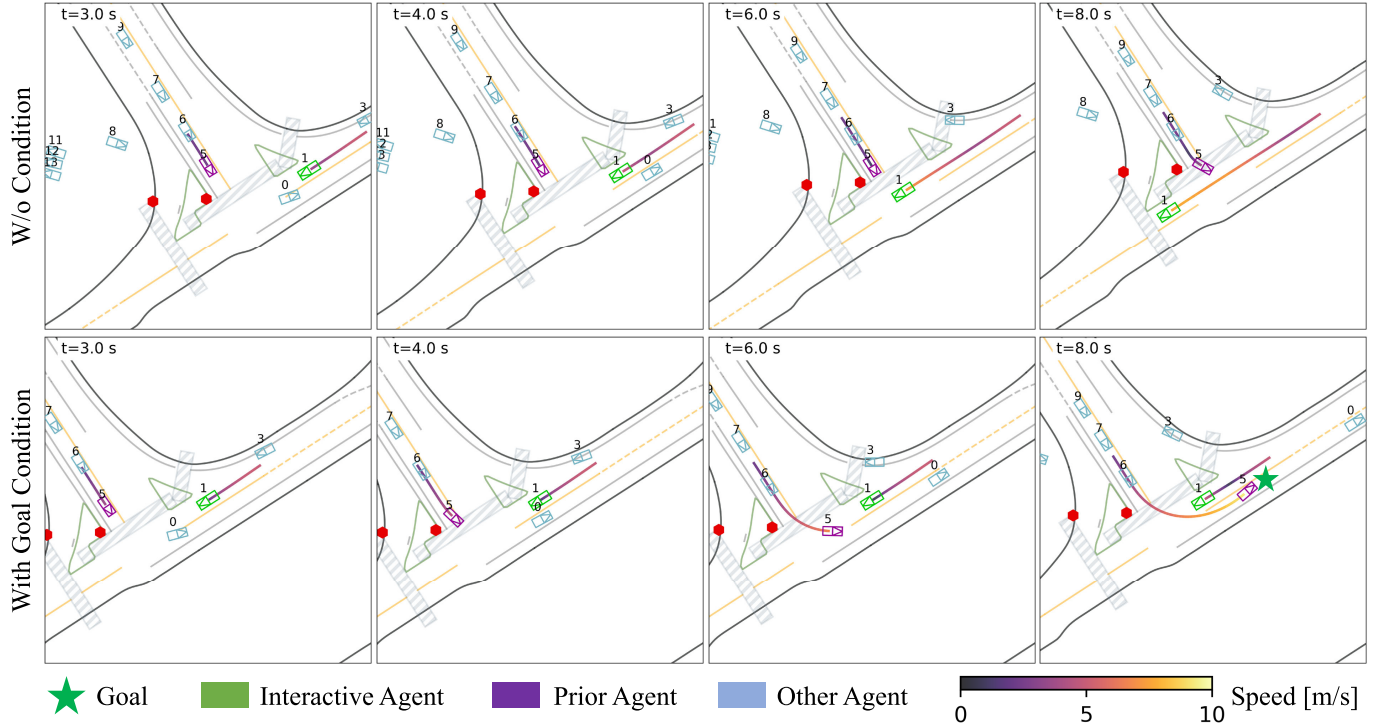


Fig. 4. VBD produces scene-consistent traffic scenarios when target agents are conditioned on specific behavior priors. **Top:** Nominal VBD rollout without guidance generates a scene-consistent scenario, where Vehicle 5 (in purple) waits at the stop sign and then proceeds. **Bottom:** Using goal-guided diffusion to minimize Vehicle 5's final position w.r.t to a given goal, we enforce Vehicle 5 to run the stop sign. VBD model generates a scene-consistent scenario with Vehicle 1 yielding to Vehicle 5.

TABLE II

SIMULATION RESULTS OF SCENARIO EDITING WITH MARGINAL PREDICTOR AND DIFFUSION POLICY OVER THREE RANDOM SEEDS

Method	Collision [%] ↓	Off-road [%] ↓	Wrong-way [%] ↓	Kin. [%] ↓	ADE [m] ↓
Marginal Prediction	5.61±0.27	6.19±0.05	0.86±0.09	0.31±0.02	1.113±0.012
Diffusion	2.47±0.09	1.21±0.14	0.57±0.02	0.24±0.01	1.010±0.007
Post-Optimization via Diffusion	2.23±0.15	1.26±0.10	0.52±0.11	0.32±0.01	0.974±0.005

vidual agents but will result in scene inconsistency if marginal trajectories are naively combined. This is because the predictor alone cannot ensure the coherence necessary for realistic multi-agent scenario generation. The denoiser in VBD can be used as an effective scenario optimizer and produce diverse and scene-consistent scenarios by first selecting goal positions for some target agents and generating joint multi-agent trajectories matching individual goals using guided sampling. Consider a scenario where we need to specify desired behaviors for A' agents, we can heuristically determine the target behaviors or goals for each target agent, represented by $\{g^i\}_{i=1:A'}$. The guidance function with behavior priors is:

$$\mathcal{J}_{goal} = - \sum_{i=1:A'} \mathcal{SL}_1(g^i - x_T^i), \quad (20)$$

where $\mathcal{SL}_1$ denotes the Smooth L1 loss, x_T^i is the state of an agent derived from actions using a differentiable dynamic function, and T is the planning horizon. The other agents in the scene will not be directly influenced by the guidance.

Case Study 1: Scene-consistent Scenario Editing: We evaluate the effectiveness of VBD as a scenario editing and optimization tool to generate scene-consistent interactions

based on marginal behavior priors for target agents. Specifically, for each target agent, we select its most likely predicted trajectory as the goal. The baseline method (marginal prediction) directly executes the trajectory for target agents. For the post-optimization with diffusion, these trajectories serve as goal priors, and we employ guided sampling to minimize the mean L_2 distance between the diffusion results and the selected goals. We compare their performance in generating specific behaviors (goal reaching) and test each method with three different random seeds. The results in Table II indicate that the diffusion policy enhances scene consistency in the generated scenarios, given specific behaviors from marginal behavior priors. It significantly reduces the collision rate and better captures interactions between agents in the scenes for goal-reaching behaviors of target agents. Moreover, even when priors are selected from suboptimal samples, e.g., off-road or wrong-way, VBD can alleviate these cases and generate scenarios that conform to the scene context.

Fig. 4 showcases the VBD model's capability in scene-consistent scenario editing, where one agent is assigned a target point based on marginal priors. In this setup, a human user oversees the simulation, manually selecting the target

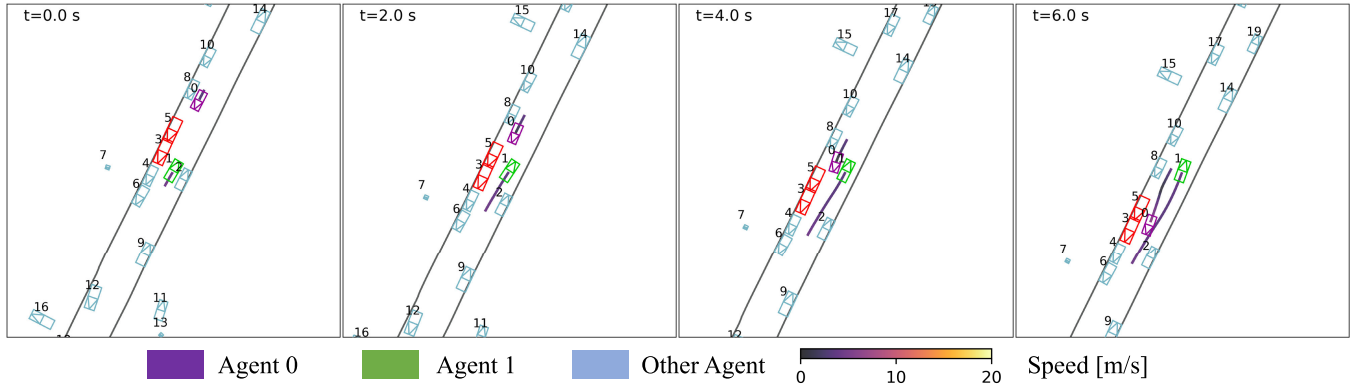


Fig. 5. Composition of collision avoidance cost-based objective in diffusion can improve generation quality. Two vehicles dynamically interact and coordinate in a narrow passage scenario with collision cost function guidance. (Note: Vehicles 3 and 5 have been in collision since the initial step.)

TABLE III
SIMULATION RESULTS OF COLLISION AVOIDANCE COST-GUIDED DIFFUSION OVER THREE RANDOM SEEDS

Method	Collision [%] ↓	Off-road [%] ↓	Wrong-way [%] ↓	Kin. [%] ↓	ADE [m] ↓
VBD	2.47±0.09	1.21±0.14	0.57±0.02	0.24±0.01	1.010±0.007
VBD + Collision Avoid	1.11±0.23	1.15±0.17	0.60±0.12	0.25±0.01	1.113±0.010

agent and its goal. By conditioning on the behavior that vehicle 5 makes a left turn, ignoring the stop sign, our VBD model's diffusion policy can generate a scenario such that vehicle 1 is compelled to yield to the other vehicle. Conversely, the nominal diffusion policy rollout without guidance follows a completely different but still scene-consistent traffic ordering.

B. Composition With Cost Functions

Incorporating collision avoidance guidance into the diffusion generation process can enhance the quality and realism of the generated scenarios and create nuanced, collision-avoiding behaviors among agents. This is implemented through a differentiable cost function, expressed as follows:

$$\mathcal{J}_{overlap} = \sum_{t=1}^T \sum_{i,j} d_{ij}(\mathbf{x}_t) \mathbb{1}(d_{ij}(\mathbf{x}_t) < \epsilon_d), \quad (21)$$

where d_{ij} represents the Minkowski distance between the footprints of agents i and j at time t . The parameter ϵ_d is the threshold for defining a potential collision.

Further, to modify the driving behavior of a target agent (i.e., to make it more aggressive), we introduce a rush cost function that penalizes speed reduction, which is:

$$\mathcal{J}_{rush} = - \sum_{t=1}^T \|a_t^i\|^2 \mathbb{1}(a_t^i < 0), \quad (22)$$

where a_t^i represents acceleration action of agent i at time t . By adding this function and the collision avoidance function together, we can implicitly modify the driving behavior of a specific agent within traffic flows.

Case Study 2: Enhancing Generation Quality: Since neural networks still exhibit unpredictability in generation, they may be prone to errors in some scenarios (e.g., off-road or collision behaviors). We aim to use a collision avoidance cost-based

optimization objective in guided sampling to further improve the performance and generation quality of VBD. Specifically, we introduce a collision avoidance cost to maximize each agent's minimum distance from others across the horizon. As shown in Table III, this collision avoidance objective effectively reduces the collision rate. The qualitative results in Fig. 5 further illustrate that incorporating a cost-based objective enables the model to generate collision-free interactions in the challenging narrow-passage scenario.

Case Study 3: Modifying Driving Style: In simulation settings, it is often desirable to alter the driving behaviors of certain agents to generate different driving styles. To achieve this, we combine a collision avoidance cost and a rush cost to change the driving behavior of a target agent in the scene generation process using guided diffusion. This composite cost guidance can effectively change the behaviors of specific agents to show aggressive driving styles, while the VBD model ensures that surrounding agents remain responsive to these behavioral changes. We manually select a target agent in a scenario and consistently apply the guidance to change the driving style of the agent in closed-loop simulations. A qualitative example of modifying an agent's driving style is illustrated in Fig. 6, where we modify the driving style of Agent 1 to exhibit an aggressive U-turn maneuver. Concurrently, the VBD model generates scene-consistent behaviors for Agent 2, who adapts by yielding to the aggressive agent.

VIII. GENERATION BEYOND DATA DISTRIBUTION

A crucial use case of simulations is to create safety-critical scenarios to stress test the robustness of AV planning systems, which must be both realistic and reactive. We employ two guidance approaches (conflict prior guided diffusion and game-theoretic guided diffusion) in our VBD model to achieve this. Examples of the generated safety-critical and adversarial

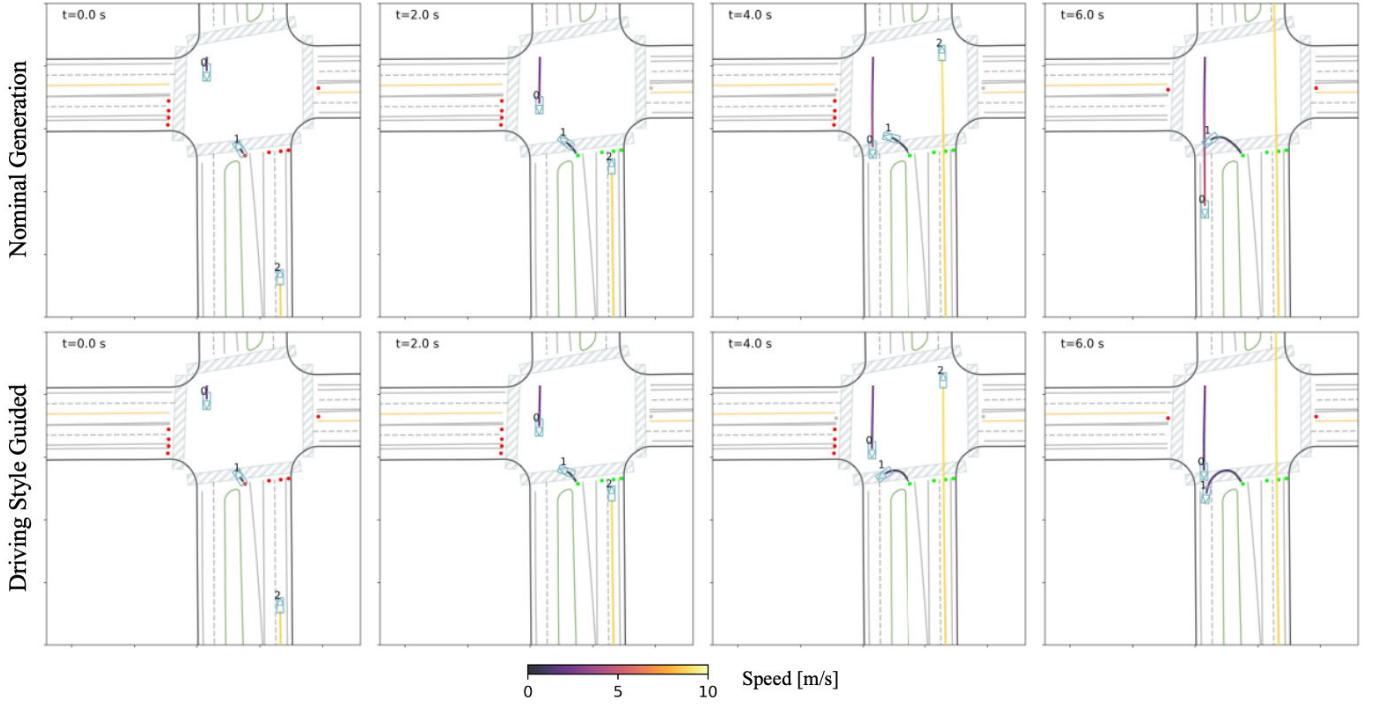


Fig. 6. Composition of rush cost-based objective in diffusion can effectively change the driving style of an agent. **Top:** Nominal VBD rollout without guidance, where Vehicle 1 yields to Vehicle 2 before the U-turn maneuver. **Bottom:** Using rush cost-guided diffusion on Vehicle 1 makes it more aggressive, forcing Vehicle 2 to slow down and yield.

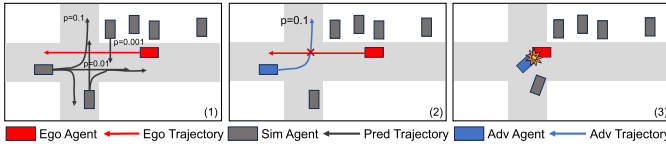


Fig. 7. Illustration of conflict prior-guided safety-critical scenario generation. (1) The behavior predictor generates multi-modal trajectories for each sim agent; (2) From the prior distribution, select the agent behavior with the highest probability of conflicting with the most likely ego plan; (3) Generate the expected scenario using the diffusion policy.

scenarios using our VBD model, implemented with these two composition methods, are showcased on the Project Website. These examples demonstrate the efficacy of our methods in generating realistic scenarios that introduce collision risks for the AV, without requiring risk-specific driving datasets or any retraining or fine-tuning of the model.

A. Safety-Critical Scenario Generation via Conflict Priors

We can utilize conflict-prior guidance to facilitate the generation of safety-critical scenarios. The behavior prior prediction significantly aids in identifying potentially unsafe agents, which is difficult for existing methods, and enhances the realism of the unsafe behaviors. By using the behavior predictor in our VBD model, we can obtain prior distributions of the possible movements of surrounding agents. From this distribution, agents that could conflict with the ego vehicle's plans are identified, and these selected priors are then used to guide the diffusion policy to generate expected scenarios. The

process of selecting conflicting agents is represented as:

$$\hat{\mathbf{x}}^{adv} = \arg \max_{i,j} \hat{\omega}_j^i [\text{col}(\hat{\mathbf{x}}_j^i, \hat{\mathbf{x}}_*^{ego}) \mathbf{1}(\hat{\omega}_j^i > \epsilon_p)], \quad \forall i \neq ego, \quad (23)$$

where $\hat{\mathbf{x}}_j^i, \hat{\omega}_j^i$ denote the trajectory and probability estimation for the prior mode j of agent i , and $\hat{\mathbf{x}}_*^{ego}$ is the most-likely prior mode of the ego vehicle. col is a collision probability function, which decreases linearly with the temporal proximity to a collision, and ϵ_p is a threshold controlling the possibility.

The core idea is to select the highest posterior probability of other agents' behaviors that conflict with the ego agent's normal driving behavior. The selected conflicting behaviors are then fed into the diffusion policy as guidance inputs. Meanwhile, the behaviors of other agents are controlled by the policy without guidance to respond to that situation. Notably, utilizing prior guidance in the diffusion policy results in more realistic safety-critical behavior compared to direct trajectory rollout, as well as robustness against selected priors that are less likely in real-world conditions. The proposed conflict prior guidance for safety-critical simulation is illustrated in Fig. 7, and generated safety-critical scenarios are presented in Fig. 8.

B. Safety-Critical Generation via Game-Theoretic Guidance

Conflict prior method is an effective sampling approach when agents' paths intersect with each other. However, this method may fail to capture the potential safety-critical scenario when no collision pairs can be automatically selected from behavior priors (e.g., the bottom row of Fig. 8). To simulate interactions in more extreme conditions, such as adversarial driving behaviors, we model the interaction between two agents as a map-constrained pursuit-evasion game, where the

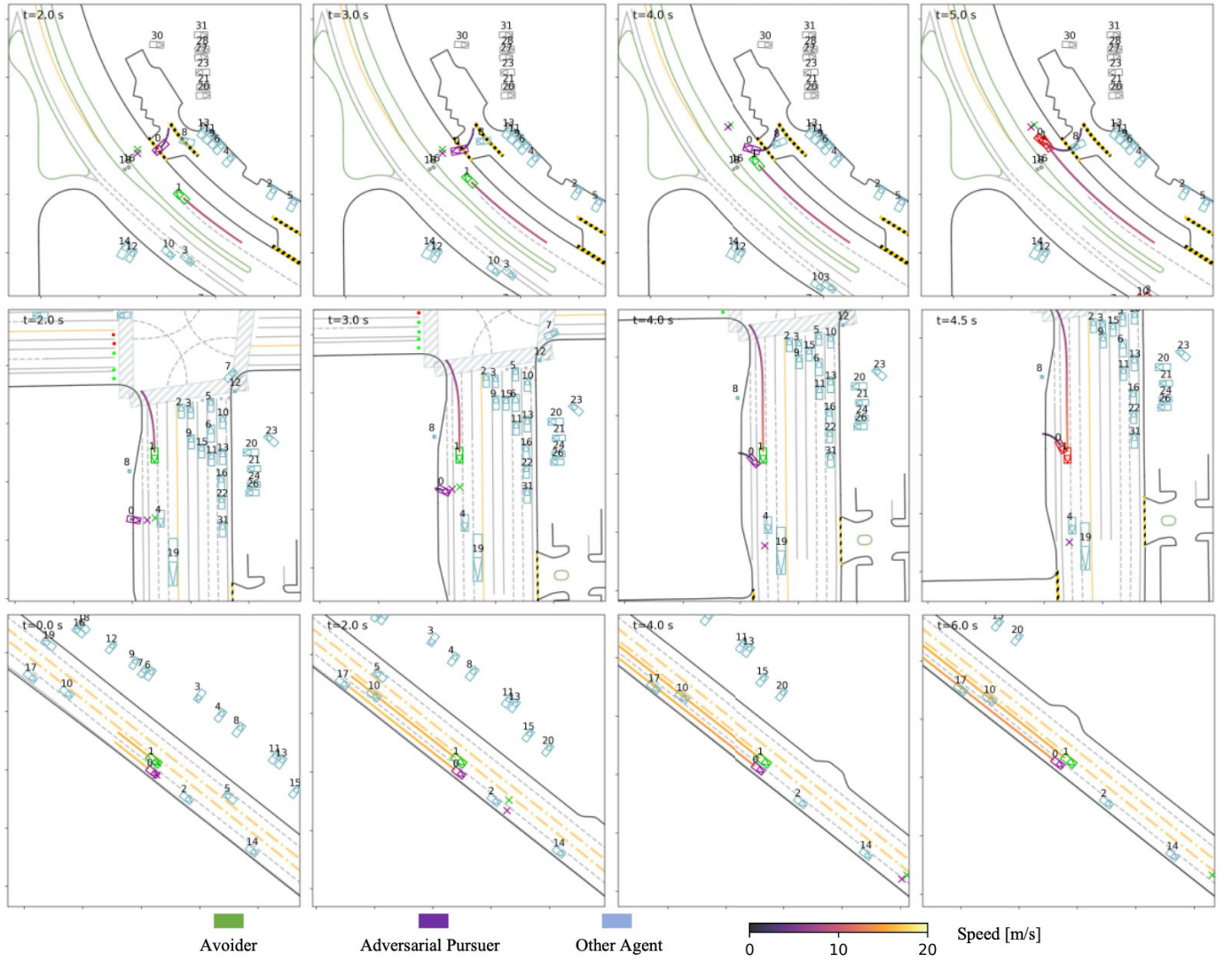


Fig. 8. Safety-critical scenario generation through conflict priors guidance. **Top and Middle:** By selecting the highest posterior probability of adversarial agents' trajectories that conflict with the ego agent's most likely prediction, we can guide the denoising process to generate safety-critical scenarios. **Bottom:** Failure case of conflict prior guidance when there were no collision pairs from behavior priors.

pursuer aims to cause a collision with the evader and stay within the road boundary, while the evader attempts to avoid it. An effective approach to solving this game is through iterative best response (IBR), such as gradient descent-ascent (GDA). Specifically, we can apply τ -GDA with time-scale separation to guarantee local convergence to a stable *minimax equilibrium* of the pursuit-evasion game [66]. We update the pursuer more frequently than the evader to provide an information advantage to the adversarial agents.

Generating a scenario with a game-theoretic solver often leads to unrealistic results. Instead, leveraging realistic traffic behaviors modeled by VBD, we propose a *game-theoretic-guided diffusion* approach (Algorithm 2). This method involves alternating denoising steps, performing gradient descent and ascent for agents, and updating the noised $\tilde{\mathbf{u}}$ until convergence. We can further enhance the realism of the scenario with *optional* gradient masks M_e and M_p , which allow us to adjust the adversity of the pursuer and the responsiveness

of the evader by selectively updating gradients at certain timesteps.

We illustrate the generation of adversarial driving scenarios with the proposed game-theoretic guidance in various scenes in Fig. 9, where the proposed guidance can generate highly interactive maneuvers in both traffic merge and lane change scenarios. In contrast to previous works [16], [17], [45], which primarily sample adversarial behaviors based on fixed trajectories of the pursued vehicle, our approach proactively optimizes the actions of both the pursuer and evader. This results in highly interactive scenarios, enriching the realism of adversarial encounters. Our proposed method facilitates the generation of highly realistic scenarios, especially with regard to adversarial behavior, by ensuring that they are proportionately challenging and reactive to the maneuvers of the ego vehicle. This strategy overcomes the shortcomings of previous methods, which tend to generate unrealistically aggressive adversarial strategies.

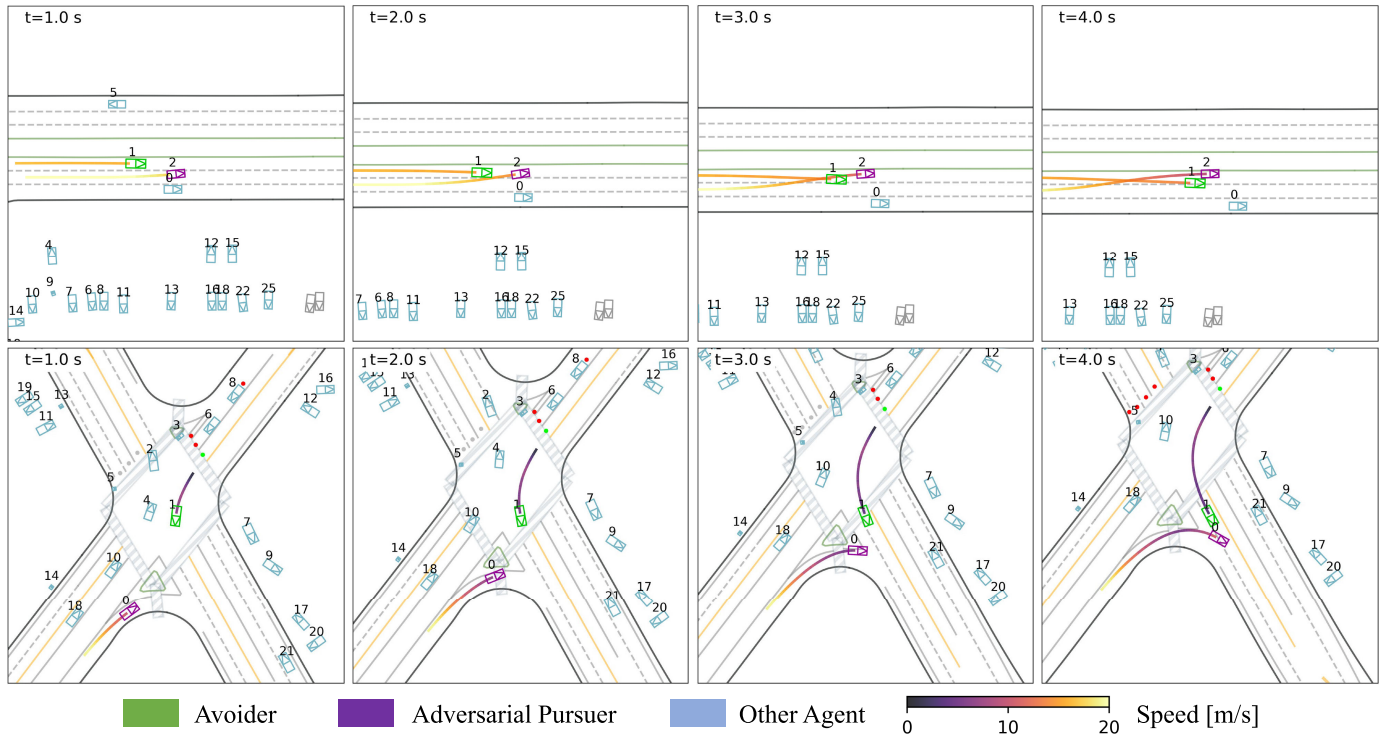


Fig. 9. Results of game-theoretic adversarial behavior generation. **Top:** The adversarial pursuer merges in front of the evader, performs a brake check, and attempts to cause a rear-end crash. The evader immediately switches its lane and avoids the collision. **Bottom:** The adversarial pursuer merges aggressively into the adjacent lane, and the evader yields to the pursuer by slowing down.

TABLE IV
IMPACT OF HISTORY ENCODING AND INTEGRATING BEHAVIOR PRIOR PREDICTION IN VBD

Predictor	No History	Collision [%] ↓	Off-road [%] ↓	ADE [m] ↓	FDE [m] ↓	minADE [m] ↓	minFDE [m] ↓
✓	×	7.44	4.44	4.23	17.94	3.55	14.57
×	✓	3.33	2.42	1.46	5.22	0.99	3.14
✓	✓	2.50	1.96	1.42	5.15	0.94	3.01

IX. ABLATION STUDIES

A. Influence of History Encoding

Causal confusion presents a considerable obstacle when applying imitation learning-based models to closed-loop simulations. Consistent with prior studies [60], [67], [68], our findings suggest that discarding historical information from agent representations and focusing solely on their current states substantially alleviates this problem. Specifically, we employ an aggressive dropout rate (50%) for the past agent trajectories at training time and only supply the current joint state to the model during inference. As shown in Table IV, experimental results reveal that models trained and tested with history dropout surpass those using full historical states encoding across all metrics, underscoring its effectiveness in mitigating causal confusion. Specifically, we observe that the baseline with full past trajectories overly accelerates or decelerates in closed-loop simulation and leads to significantly high displacement errors, highlighting its copying behavior through kinematic history. By adopting history dropout, we achieve enhanced performance in closed-loop simulations, highlighting the significance of suitable state representation.

B. Integration of Behavior Prediction

Another key design choice in VBD is the integration of a multi-modal behavior predictor within the model architecture as a co-training task. The results shown in Table IV indicate that the inclusion of the behavior predictor significantly enhances performance in closed-loop simulation tests compared to a baseline approach lacking prediction capabilities. The integration of the predictor contributes to the refinement of the diffusion policy by facilitating more effective learning within the model's encoder. This interaction strengthens the encoder's capacity to extract meaningful features, which are critical for downstream tasks. Moreover, the predictor provides behavior priors for individual agents, thereby enabling behavior editing and enhancing the controllability of the diffusion policy.

C. Noise Scheduling Design

Traditional cosine scheduling tends to favor low noise variance regions during the initial diffusion steps, potentially allowing the model to bypass proper scene representation learning by relying on shortcuts through less noisy inputs.

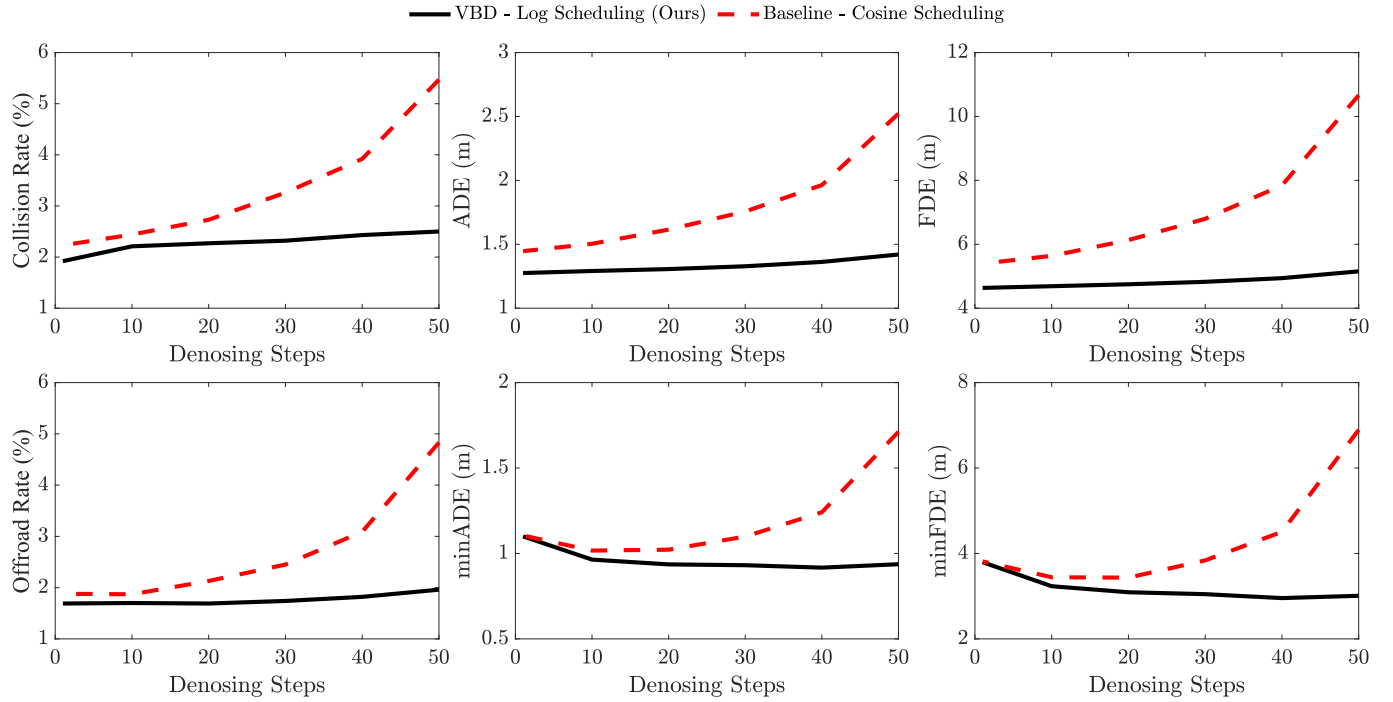


Fig. 10. Effects of noise scheduling on closed-loop simulation performance. The metrics (except for minADE and minFDE) are average values over 32 samples of closed-loop rollout. The VBD model trained with cosine scheduling exhibits significant performance degradation across all metrics. In contrast, VBD trained with the proposed log scheduling maintains low collision and off-road rates across all denoising steps. Additionally, minADE and minFDE metrics decrease as the number of denoising steps increases, reflecting the model's ability to generate scenarios closer to the ground truth. A slight increase in ADE and FDE metrics indicates improved sample diversity with more denoising steps.

Algorithm 2 Game-Theoretic-Guided Adversarial Scenario Generation With Gradient Descent-Ascent

Require: Denoiser $\mathcal{D}_\theta$, diffusion steps K , evader index e , pursuer index p , guidance steps N_g , descent steps N_e , ascent steps N_p , evader gradient mask M_e , pursuer gradient mask M_p , scaling parameter λ , standard deviation σ_k .

```

1:  $\tilde{\mathbf{u}}_K \sim \mathcal{N}(0, \mathbf{I})$   $\triangleright$ Sample initial trajectory
2: for  $k \leftarrow K$  to 1 do
3:    $\hat{\mathbf{u}} \leftarrow \mathcal{D}_\theta(\tilde{\mathbf{u}}_k, k, \mathbf{c})$   $\triangleright$ Predict denoised control sequence
4:    $\tilde{\mu}_k \leftarrow \frac{\sqrt{\alpha_k}(1-\bar{\alpha}_{k-1})}{1-\bar{\alpha}_k} \tilde{\mathbf{u}}_k + \frac{\sqrt{\bar{\alpha}_{k-1}}\beta_k}{1-\bar{\alpha}_k} \hat{\mathbf{u}}$   $\triangleright$ Calculate unguided posterior  $\tilde{\mu}_k$ 
5:   for  $i \leftarrow 1$  to  $N_g$  do
6:     for  $i_d \leftarrow 1$  to  $N_e$  do
7:        $\hat{\mathbf{x}} \leftarrow f(\mathcal{D}_\theta(\tilde{\mu}_k))$ 
8:        $\mathcal{J} \leftarrow \min_t d_{e,p}(\hat{\mathbf{x}}_t) + \min_t d_r(\hat{\mathbf{x}}_t^e)$ 
9:        $\tilde{\mu}_k^e \leftarrow \tilde{\mu}_k - \lambda \sigma_k M_e \nabla_{\tilde{\mu}_k^e} \mathcal{J}$   $\triangleright$ Gradient descent for evader
10:    end for
11:    for  $i_d \leftarrow 1$  to  $N_p$  do
12:       $\hat{\mathbf{x}} \leftarrow f(\mathcal{D}_\theta(\tilde{\mu}_k))$ 
13:       $\mathcal{J} \leftarrow \min_t d_{e,p}(\hat{\mathbf{x}}_t) + \min_t d_r(\hat{\mathbf{x}}_t^p)$ 
14:       $\tilde{\mu}_k^p \leftarrow \tilde{\mu}_k + \lambda \sigma_k M_p \nabla_{\tilde{\mu}_k^p} \mathcal{J}$   $\triangleright$ Gradient ascent for pursuer
15:    end for
16:  end for
17:   $\tilde{\mathbf{u}}_{k-1} \sim \mathcal{N}(\tilde{\mu}_k, \sigma_k^2 \mathbf{I})$   $\triangleright$ Sample previous-step noised control sequence
18: end for
19: Return: Final control sequence  $\mathbf{u} \leftarrow \tilde{\mathbf{u}}_0$ 

```

Consequently, when encountering out-of-distribution actions during testing, the model struggles to adapt and instead produces degraded output by directly passing the actions through the final denoising steps. This insight motivated us to develop a logarithmic noise schedule, presented in Eq. (15), which maintains a suitable signal-to-noise ratio (SNR) throughout the diffusion process, ensuring effective action denoising that is dependent on scene context. Additionally, experimental results in Fig. 10 validate the efficacy of our proposed approach, showcasing improved trajectory diffusion performance under the new noise schedule. By adopting the log scheduling strategy, VBD successfully generates diverse scenarios without compromising generation quality.

The result in Fig. 10 reveals that the generation quality is satisfactory after just the first denoising step, especially regarding collision rate, off-road rate, ADE, and FDE. Further denoising steps seem to degrade these metrics. One possible explanation is that traffic behavior diffusion is heavily influenced by scene context; hence, the initial denoising step can yield plausible outcomes that adhere to both agent interactions and map constraints. Nevertheless, subsequent denoising steps play a vital role in striking a balance between fidelity and diversity. With increasing numbers of steps, we notice a decline in the minADE and minFDE metrics, suggesting that one of the generated samples draws closer to the ground truth. Average ADE and FDE metrics have increased, highlighting greater sample diversity. This outcome demonstrates that additional denoising steps enable the model to explore a wider range of feasible outcomes while preserving high-quality trajectories. Furthermore, these extra steps are crucial

TABLE V
CLOSED-LOOP SIMULATION RESULTS WITH DDIM SAMPLING

Step	Unguided Scenario Generation				Guided Scenario Generation			
	Collision [%] ↓	Off-road [%] ↓	ADE [m] ↓	Runtime [s] ↓	Collision [%] ↓	Off-road [%] ↓	ADE [m] ↓	Runtime [s] ↓
5	2.26	1.62	1.23	0.16 ± 0.023	1.01	1.14	1.32	0.55 ± 0.021
10	2.39	1.72	1.25	0.31 ± 0.001	1.06	1.35	1.35	1.19 ± 0.034
25	2.41	1.77	1.29	0.73 ± 0.002	1.01	1.34	1.54	2.87 ± 0.078
DDPM	2.50	1.96	1.42	1.48 ± 0.005	1.11	1.58	1.68	6.33 ± 0.130

for facilitating control and guidance over agent behaviors, particularly in applications such as traffic scenario generation.

D. Accelerating Inference

Employing Denoising Diffusion Implicit Models (DDIM) [36] can significantly accelerate the inference process of the VBD model while maintaining performance. To evaluate the impact of diffusion steps on simulation and runtime performance, we conduct experiments with varying numbers of diffusion steps in DDIM, and the detailed results are summarized in Table V. The runtime of each inference step is measured on a system equipped with an AMD 7950X CPU and an NVIDIA RTX 4090 GPU. The results reveal that the runtime grows linearly with the number of diffusion steps. Notably, employing 5 or 10 diffusion steps achieves a favorable balance between generation quality and real-time performance (with a feasible replan frequency of 1 Hz). Especially when combined with collision cost guidance, we can significantly reduce the collision rate and enhance the quality of the generated behaviors. Specifically, using 5 diffusion steps in DDIM guided sampling yields a notable reduction in collision rates and improves scenario realism without compromising computational efficiency. Moreover, increasing the number of diffusion steps results in diminishing returns for collision avoidance and a negative impact on the log divergence (ADE) metric. This suggests that the performance and efficiency of VBD can be further enhanced with DDIM sampling to balance computational efficiency and performance while retaining a certain level of controllability.

X. CONCLUSION

We present Versatile Behavior Diffusion (VBD), a traffic scenario generation framework that leverages diffusion models to produce realistic, diverse, and controllable multi-agent interactions in closed-loop simulation. Our method demonstrates strong performance in multi-agent behavior simulation and effectively captures complex interaction patterns. A key strength of VBD lies in its flexible inference-time control, which enables user-guided scenario editing and behavior conditioning to meet specific requirements. Through extensive experiments, we show that VBD is effective across a broad range of traffic simulation tasks, including safety-critical scenarios. Several directions remain for future work. A key next step is integrating VBD into AV planning and testing pipelines to enable systematic evaluation and improvement

of planning algorithms under generated scenarios, including rare and adversarial cases. Enhancing runtime efficiency will also be critical for real-time deployment in AV simulation and training systems. Additionally, incorporating large language models (LLMs) for context-aware scenario generation and extending the framework to diverse traffic domains, such as highways or urban mixed-autonomy settings, are promising avenues for further development.

REFERENCES

- [1] Q. Li, Z. Peng, L. Feng, Q. Zhang, Z. Xue, and B. Zhou, "MetaDrive: Composing diverse driving scenarios for generalizable reinforcement learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 3, pp. 3461–3475, Mar. 2023.
- [2] M. Treiber, A. Hennecke, and D. Helbing, "Congested traffic states in empirical observations and microscopic simulations," *Phys. Rev. E, Stat. Phys. Plasmas Fluids Relat. Interdiscip. Top.*, vol. 62, no. 2, pp. 1805–1824, Aug. 2000.
- [3] A. Kesting, M. Treiber, and D. Helbing, "General lane-changing model MOBIL for car-following models," *Transp. Res. Rec., J. Transp. Res. Board*, vol. 1999, no. 1, pp. 86–94, Jan. 2007.
- [4] S. Shi, L. Jiang, D. Dai, and B. Schiele, "Motion transformer with global intention localization and local movement refinement," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 6531–6543.
- [5] M. Igl et al., "Symphony: Learning realistic and diverse agents for autonomous driving simulation," in *Proc. Int. Conf. Robot. Autom. (ICRA)*, May 2022, pp. 2445–2451.
- [6] N. Nayakanti, R. Al-Rfou, A. Zhou, K. Goel, K. S. Refaat, and B. Sapp, "Wayformer: Motion forecasting via simple & efficient attention networks," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2023, pp. 2980–2987.
- [7] Z. Zhang, A. Liniger, D. Dai, F. Yu, and L. Van Gool, "TrafficBots: Towards world models for autonomous driving simulation and motion prediction," 2023, *arXiv:2303.04116*.
- [8] L. Feng, Q. Li, Z. Peng, S. Tan, and B. Zhou, "TrafficGen: Learning to generate diverse and realistic traffic scenarios," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2023, pp. 3567–3575.
- [9] H. Caesar et al., "nuScenes: A multimodal dataset for autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 11621–11631.
- [10] B. Wilson et al., "Argoverse 2: Next generation datasets for self-driving perception and forecasting," 2023, *arXiv:2301.00493*.
- [11] N. Montali et al., "The Waymo open sim agents challenge," 2023, *arXiv:2305.12032*.
- [12] Y. Chen, B. Ivanovic, and M. Pavone, "ScePT: Scene-consistent, policy-based trajectory predictions for planning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 17103–17112.
- [13] H. X. Liu and S. Feng, "Curse of rarity for autonomous vehicles," *Nature Commun.*, vol. 15, no. 1, p. 4808, Jun. 2024.
- [14] Y. Cao, C. Xiao, A. Anandkumar, D. Xu, and M. Pavone, "AdvDO: Realistic adversarial attacks for trajectory prediction," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 36–52.
- [15] N. Hanselmann, K. Renz, K. Chitta, A. Bhattacharyya, and A. Geiger, "KING: Generating safety-critical driving scenarios for robust imitation via kinematics gradients," in *Proc. Eur. Conf. Comput. Vis. Cham, Switzerland: Springer*, 2022, pp. 335–352.

- [16] D. Rempe, J. Philion, L. J. Guibas, S. Fidler, and O. Litany, "Generating useful accident-prone driving scenarios via a learned traffic prior," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jul. 2022, pp. 17305–17315.
- [17] L. Zhang, Z. Peng, Q. Li, and B. Zhou, "CAT: Closed-loop adversarial training for safe end-to-end driving," in *Proc. Conf. Robot Learn.*, 2023, pp. 2357–2372.
- [18] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli, "Deep unsupervised learning using nonequilibrium thermodynamics," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 2256–2265.
- [19] Y. Song and S. Ermon, "Generative modeling by estimating gradients of the data distribution," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 1–13.
- [20] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," 2020, *arXiv:2011.13456*.
- [21] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. NIPS*, vol. 33, 2020, pp. 6840–6851.
- [22] S. Suo, S. Regalado, S. Casas, and R. Urtasun, "TrafficSim: Learning to simulate realistic multi-agent behaviors," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 10395–10404.
- [23] D. Xu, Y. Chen, B. Ivanovic, and M. Pavone, "BITS: Bi-level imitation for traffic simulation," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2023, pp. 2929–2936.
- [24] W.-J. Chang, C. Tang, C. Li, Y. Hu, M. Tomizuka, and W. Zhan, "Editing driver character: Socially-controllable behavior generation for interactive traffic simulation," 2023, *arXiv:2303.13830*.
- [25] J. Philion, X. B. Peng, and S. Fidler, "Trajenglish: Traffic modeling as next-token prediction," 2023, *arXiv:2312.04535*.
- [26] S. Sun et al., "DriveSceneGen: Generating diverse and realistic driving scenarios from scratch," 2023, *arXiv:2309.14685*.
- [27] Z. Huang, X. Mo, and C. Lv, "Multi-modal motion prediction with transformer-based neural network for autonomous driving," in *Proc. Int. Conf. Robot. Autom. (ICRA)*, May 2022, pp. 2605–2611.
- [28] Z. Zhou, J. Wang, Y.-H. Li, and Y.-K. Huang, "Query-centric trajectory prediction," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 17863–17873.
- [29] X. Mo, Z. Huang, Y. Xing, and C. Lv, "Multi-agent trajectory prediction with heterogeneous edge-enhanced graph attention network," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 7, pp. 9554–9567, Jul. 2022.
- [30] S. Shi, L. Jiang, D. Dai, and B. Schiele, "MTR++: Multi-agent motion prediction with symmetric scene modeling and guided intention querying," 2023, *arXiv:2306.17770*.
- [31] Z. Huang, H. Liu, and C. Lv, "GameFormer: Game-theoretic modeling and learning of transformer-based interactive prediction and planning for autonomous driving," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 3880–3890.
- [32] C. Jiang, A. Cornman, C. Park, B. Sapp, Y. Zhou, and D. Anguelov, "MotionDiffuser: Controllable multi-agent motion prediction using diffusion," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 9644–9653.
- [33] M. Niedoba et al., "A diffusion-model of joint interactive navigation," in *Proc. 37th Conf. Neural Inf. Process. Syst.*, 2023, pp. 1–17.
- [34] A. Seff et al., "MotionLM: Multi-agent motion forecasting as language modeling," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 8545–8556.
- [35] Z. Zhou et al., "BehaviorGPT: Smart agent simulation for autonomous driving with next-patch prediction," 2024, *arXiv:2405.17372*.
- [36] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," 2020, *arXiv:2010.02502*.
- [37] L. Zhang and M. Agrawala, "Adding conditional control to text-to-image diffusion models," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2023, pp. 3836–3847.
- [38] Z. Kong, W. Ping, J. Huang, K. Zhao, and B. Catanzaro, "DiffWave: A versatile diffusion model for audio synthesis," in *Proc. Int. Conf. Learn. Represent.*, 2020, pp. 1–17.
- [39] P. Esser, J. Chiu, P. Atighechian, J. Granskog, and A. Germanidis, "Structure and content-guided video synthesis with diffusion models," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 7312–7322.
- [40] Z. Guo, X. Gao, J. Zhou, X. Cai, and B. Shi, "SceneDM: Scene-level multi-agent trajectory generation with consistent diffusion models," 2023, *arXiv:2311.15736*.
- [41] Z. Zhong et al., "Guided conditional diffusion for controllable traffic simulation," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2023, pp. 3560–3566.
- [42] Z. Zhong et al., "Language-guided traffic simulation via scene-level diffusion," 2023, *arXiv:2306.06344*.
- [43] D. Rempe et al., "Trace and pace: Controllable pedestrian animation via guided trajectory diffusion," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 13756–13766.
- [44] C. Xu, D. Zhao, A. Sangiovanni-Vincentelli, and B. Li, "DiffScene: Diffusion-based safety-critical scenario generation for autonomous vehicles," in *Proc. 2nd Workshop New Frontiers Adversarial Mach. Learn.*, 2023, pp. 8797–8805.
- [45] W.-J. Chang, F. Pittaluga, M. Tomizuka, W. Zhan, and M. Chandraker, "SAFE-SIM: Safety-critical closed-loop traffic simulation with diffusion-controllable adversaries," 2023, *arXiv:2401.00391*.
- [46] D. Anguelov et al., "SceneDiffuser: Efficient and controllable driving simulation initialization and rollout," in *Proc. Adv. Neural Inf. Process. Syst.*, 37, 2024, pp. 55729–55760.
- [47] B. D. Ziebart et al., "Maximum entropy inverse reinforcement learning," in *Proc. AAAI*, vol. 8, Chicago, IL, USA, 2008, pp. 1433–1438.
- [48] Y. LeCun, S. Chopra, R. Hadsell, M. Ranzato, and F. Huang, "A tutorial on energy-based learning," *Predicting Structured Data*, vol. 1, pp. 1–59, Aug. 2006.
- [49] Y. Song and D. P. Kingma, "How to train your energy-based models," 2021, *arXiv:2101.03288*.
- [50] Y. Du and I. Mordatch, "Implicit generation and modeling with energy based models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 1–11.
- [51] N. Liu, S. Li, Y. Du, A. Torralba, and J. B. Tenenbaum, "Compositional visual generation with composable diffusion models," in *Proc. Eur. Conf. Comput. Vis. Cham, Switzerland: Springer*, 2022, pp. 423–439.
- [52] C. Chi et al., "Diffusion policy: Visuomotor policy learning via action diffusion," 2023, *arXiv:2303.04137*.
- [53] A. Q. Nichol and P. Dhariwal, "Improved denoising diffusion probabilistic models," in *Proc. 38th Int. Conf. Mach. Learn.*, Jul. 2021, pp. 8162–8171.
- [54] P. Dhariwal and A. Nichol, "Diffusion models beat GANs on image synthesis," in *Proc. NIPS*, vol. 34, 2021, pp. 8780–8794.
- [55] M. Janner, Y. Du, J. Tenenbaum, and S. Levine, "Planning with diffusion for flexible behavior synthesis," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 9902–9915.
- [56] Z. Huang, P. Karkus, B. Ivanovic, Y. Chen, M. Pavone, and C. Lv, "DTPP: Differentiable joint conditional prediction and cost evaluation for tree policy planning in autonomous driving," 2023, *arXiv:2310.05885*.
- [57] S. Ettinger et al., "Large scale interactive motion forecasting for autonomous driving: The Waymo open motion dataset," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2021, pp. 9710–9719.
- [58] M. Chang et al., "Argoverse: 3D tracking and forecasting with rich maps," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2019, pp. 8748–8757.
- [59] C. Gulino et al., "Waymax: An accelerated, data-driven simulator for large-scale autonomous driving research," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 7730–7742.
- [60] J. Cheng, Y. Chen, X. Mei, B. Yang, B. Li, and M. Liu, "Rethinking imitation-based planners for autonomous driving," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2024, pp. 14123–14130.
- [61] W. Wu, X. Feng, Z. Gao, and Y. Kan, "SMART: Scalable multi-agent real-time motion generation via next-token prediction," 2024, *arXiv:2405.15677*.
- [62] Y. Hu et al., "Solving motion planning tasks with a scalable generative model," 2024, *arXiv:2407.02797*.
- [63] Y. Wang, T. Zhao, and F. Yi, "Multiverse transformer: 1st place solution for waymo open sim agents challenge 2023," 2023, *arXiv:2306.11868*.
- [64] Z. Zhang, C. Sakaridis, and L. Van Gool, "TrafficBots V1.5: Traffic simulation via conditional VAEs and transformers with relative pose encoding," 2024, *arXiv:2406.10898*.
- [65] S. Sadat, J. Buhmann, D. Bradley, O. Hilliges, and R. M. Weber, "CADS: Unleashing the diversity of diffusion models through condition-annealed sampling," 2023, *arXiv:2310.17347*.
- [66] T. Fiez, L. Ratliff, E. Mazumdar, E. Faulkner, and A. Narang, "Global convergence to local minmax equilibrium in classes of nonconvex zero-sum games," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 29049–29063.
- [67] P. De Haan, D. Jayaraman, and S. Levine, "Causal confusion in imitation learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 1–12.
- [68] M. Bansal, A. Krizhevsky, and A. Ogale, "ChauffeurNet: Learning to drive by imitating the best and synthesizing the worst," 2018, *arXiv:1812.03079*.



Zhiyu Huang (Member, IEEE) received the B.E. degree from Chongqing University, China, in 2019, and the Ph.D. degree from Nanyang Technological University, Singapore, in 2024. He is currently a Post-Doctoral Researcher with the University of California, Los Angeles. His research interests include learning-enabled autonomous driving, multi-agent behavior modeling, reinforcement learning, and generative models for decision-making and simulation. He has received multiple research awards, including best paper and benchmark challenge awards in

autonomous driving. He currently serves as an Associate Editor for IEEE OPEN JOURNAL OF INTELLIGENT TRANSPORTATION SYSTEMS.



Zixu Zhang received the B.S. degree in mechanical engineering from the University of Illinois Urbana-Champaign, Champaign, IL, USA, in 2018, and the M.S. degree in robotics from the University of Michigan, Ann Arbor, MI, USA, in 2020. He is currently pursuing the Ph.D. degree in electrical and computer engineering with Princeton University, Princeton, NJ, USA.



Ameya Vaidya received the bachelor's degree in computer science from Princeton University in Spring 2024. During his undergraduate studies, he developed strong foundations in machine learning, software engineering, and data-driven systems, and engaged in early research experiences in machine learning and data science.



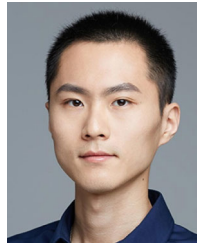
Yuxiao Chen received the B.S. degree from Tsinghua University, China, in 2013, and the Ph.D. degree from the University of Michigan, Ann Arbor, MI, USA, in 2018. He is currently a Senior Research Scientist with the Autonomous Vehicle Research Group, NVIDIA. Prior to joining NVIDIA in 2021, he spent three years as a Post-Doctoral Researcher with California Institute of Technology. His research focuses on planning and decision-making for safety-critical autonomous and multi-agent systems, with applications to autonomous vehicles, ground and

aerial robots, and power networks. His current work emphasizes trajectory prediction and contingency planning for autonomous driving systems.



Jaime Fernandez Fisac (Member, IEEE) received the Engineering degree from the Universidad Politécnica de Madrid, the master's degree in aeronautics from Cranfield University, and the Ph.D. degree in electrical engineering and computer sciences from the University of California, Berkeley. He is an Assistant Professor of electrical and computer engineering with Princeton University, where he directs the Safe Robotics Laboratory and co-directs Princeton AI4ALL. Before joining Princeton, he was a Research Scientist with Waymo, where he

pioneered new approaches to interaction planning that continue to shape how autonomous vehicles share the road today. He is also the Co-Founder of Vault Robotics, a startup developing agile delivery robots that work alongside human drivers. His research integrates control systems, game theory, and artificial intelligence to equip robots with transparent safety assurances that users and the public can trust.



Chen Lv (Senior Member, IEEE) received the joint Ph.D. degree from the Department of Automotive Engineering, Tsinghua University, China, and the EECS Department, University of California, Berkeley, USA, in January 2016. He is an Associate Professor with the School of Mechanical and Aerospace Engineering, Nanyang Technological University (NTU), Singapore. He also holds joint appointments as the Research Director of NTU, Vice President Research Office, the Director of M.Sc. degree in robotics and intelligent systems, a Thrust

Lead in AI with the Continental-NTU Corporation Laboratory, and the Cluster Director in Future Mobility Solutions with ERI@N. He was as a Research Fellow with the Advanced Vehicle Engineering Center, Cranfield University, U.K., from 2016 to 2018. He joined NTU, as a Nanyang Assistant Professor and has founded the Automated Driving and Human-Machine System (AutoMan) Research Laboratory in June 2018. His research focuses on advanced vehicle control and intelligence, where he has contributed four books, over 200 papers, and received 12 granted patents and five patent applications.